System Design Patterns — Interview Edition

The book you re-read the night before a system-design interview: all 18 modules of *System Design Patterns* — 60+ patterns, 5 added lessons and 29 case studies — cut down to compact pattern and case cards, the numbers, the likely questions and the traps. Nothing here is new — every line comes from the full notes; open the matching module there when you need the incident, the walkthrough or the diagram explained.

How to use this edition

- **One section per module (1–18)**, always in the same order: **Must-know** (key terms, then one compact card per pattern: *Problem* → *How* → *Trade-offs* → *Real systems* → *One-liner*; for the case-study modules 13–18, one card per case: *Requirements* → *Key numbers* → *Core design* → *Hardest part* → *One-liner*) → **Key facts & numbers** (comparison tables) → **In the interview** (likely questions, short answers) → **Gotchas** (mistake → correct idea) → **TL;DR**.
- **Interview framework:** chapter 1 holds the one-page framework (numbers first, then *Move* → *Live* → *Fast* → *Fail* → *Grow*) and the back-of-envelope numbers table. Use it as the agenda you say out loud.
- **Pattern selection:** chapter 12 ends with the *symptom* → *pattern* table and the decision rules — the fastest way to pick a pattern under pressure.
- **Night before:** the Top 25 below, then every chapter's **In the interview** and **Gotchas**, then chapter 12's tables and the *Last-minute checklist*. If you know the prompt ("design a chat app"), reread its case card in chapters 13–18 and say its numbers aloud.
- **A week before:** read every card and say each one-liner aloud *with its cost*; for any card you can't explain, reopen the full module.
- The habit every chapter repeats: **never name a pattern without naming its price.**

Top 25 questions across the book

#	Question	One-line answer	Ch
1	How do you structure a design answer?	Numbers first, then Move → Live → Fast → Fail → Grow; name every cost; spend time on failure	1, 12
2	How many requests in flight / how big a pool?	Little's law: rate × duration (300/s × 2 s = 600)	1, 6
3	Sync call or queue?	Sync if the caller's next step needs the answer; queue if nobody waits — and make consumers idempotent	2
4	Queue vs pub/sub?	Queue = a job for one worker; pub/sub = a fact for every subscriber	2
5	Chat: how do you keep message order and avoid loss?	One sequencer per conversation (gap-free seq), ack after durable append, client message ID for dedupe, per-device inbox cursor	14
6	SSE or WebSocket?	SSE for one-way streams (LLM tokens); WebSocket only when the client streams back	2
7	DB overloaded — what do you do?	Ask reads or writes: replicas + cache for reads, shard for writes/size	3
8	How do you choose a shard key?	In nearly every query, spreads load evenly, keeps related rows together (e.g., customer ID)	3
9	Why consistent hashing?	Resize moves ~1/N keys instead of ~80%; vnodes keep it even; doesn't fix hot keys	3
10	How does a DB not lose acked writes?	WAL: append + fsync before ack, replay from checkpoint on restart	3
11	Cache invalidation on write?	Update DB, then DELETE the key (not SET); TTL per data type as insurance	4
12	Mobile app: does the UI read from the network?	No — UI observes the local DB, the network writes into it; outbox + idempotency key for writes; push is a hint	18
13	What if this call fails or hangs?	Timeout (2–3× p99) → jittered retries (one layer) → idempotency → breaker → degradation	5
14	Exactly-once delivery?	Impossible; at-least-once + idempotent consumer = exactly-once effect	5, 10
15	Two guests book the last hotel room — who wins?	One short transaction of conditional per-night UPDATEs (the row is the lock) + idempotency key; hold → pay → confirm	17
16	How would you scale this?	Scale up to buy time; make it stateless; LB + N+1; auto-scale on the right signal; pool connections	6
17	Distributed transaction across services?	Saga: local steps + durable log + idempotent compensations; 2PC only inside one trust boundary	7
18	Quorum math?	W + R > N (N=3, W=2, R=2) gives overlap — visibility, not ordering	7
19	Design a rate limiter	Token bucket per API key (r, b), cost weights, Redis counters, 429 + Retry-After	8
20	Gateway vs service mesh?	Gateway: north-south, user identity, thin; mesh: east-west, workload mTLS, retries, fail-static	8
21	Health checks?	Liveness never checks dependencies; readiness only essentials; deep checks → monitoring	9
22	Ship a risky release safely?	Blue-green for environment, sticky 1% canary vs concurrent baseline, flags per feature; expand-contract	9

#	Question	One-line answer	Ch
23	Keep search/cache/warehouse in sync with the DB?	Log-based CDC with tombstones + idempotent consumers; outbox for domain events; never dual-write	10
24	Survive a region outage / limit a bad deploy's blast radius?	DR strategy by RPO/RTO, data with home regions, static stability; cells deployed one at a time	6
25	Design a chatbot over company docs?	RAG: CDC-fed ingestion, ACL-filtered retrieval, rerank, cited generation via an LLM gateway	11

1. Thinking in Patterns

The method behind every later chapter: what a pattern is, the Five Questions interview framework, and back-of-envelope math.

Must-know

- **System design pattern** – a reusable solution to a recurring problem, *bundled with its cost*. Three parts: general problem + standard solution + cost. Missing any one = incomplete.
- **Technology ≠ pattern** – Redis, Kafka, Postgres *implement* patterns; they don't define them.
- **Architecture** – several patterns cooperating (Netflix ≈ fifteen). "Design X" = find the problems → pick one pattern per problem and check interactions → name every cost (and decline patterns the problem doesn't deserve).
- **Every pattern is a trade** – queue: absorbs bursts / adds latency + duplicates; cache: fast reads / stale data; sharding: write capacity / hard cross-shard queries; circuit breaker: fails fast / may trip while the dependency could still answer.
- **Weak vs strong answer** – "We'll use Kafka" vs "A queue fits because nobody waits on this work; it can redeliver, so the consumer dedupes; it can reorder, and this operation tolerates that." → **problem, pattern, cost**.
- **How interviews test patterns** – spotting problems from symptoms (all health checks green but one feature starved → *bulkhead*), pattern interactions ("could a retry charge twice?"), and defending choices ("why not X everywhere?" = "what does X cost?").
- **Five Questions** – a fixed-order checklist: **Move → Live → Fast → Fail → Grow** (arrows, databases, caches, failure budget, capacity plan). Maps to chapters 2–6.
- **Step 0** – get the numbers first: users, RPS, data size, read:write ratio, cost of an outage.
- **Little's law** – things in flight = arrival rate × duration. Sizes thread pools, connection pools, bulkheads, backlog drain time.
- **Staleness budget** – how old each type of data may be; pick the caching mechanism that keeps to it.

Key facts & numbers

The system-design interview framework (one page)

Step	Question to ask aloud	Fills in	Pattern menu (chapter)
0	Users? RPS (avg, peak)? Data size & growth? Read:write? Cost of downtime?	Requirements	Back-of-envelope (1)
1 Move	Who talks to whom? Does the caller wait? Exactly-once? Many interested services? Crosses a company boundary? Continuous flow?	The arrows	Request-response, queue, pub/sub, EDA, webhooks, SSE, WebSocket (2)
2 Live	Source of truth? Size and growth? Read:write? History or current state?	The databases	Replicas, sharding, consistent hashing, WAL, event sourcing, CQRS (3)
3 Fast	Which reads repeat? Staleness budget per data type? Hit ratio? What if the hot key expires?	The caches	Cache-aside → read/write-through, write-behind, stampede prevention (4)
4 Fail	For *every arrow*: never answers / fails once / down 10 min? Can a retry double an effect? What does the user see?	Failure budget	Timeout, retry+backoff, idempotency, circuit breaker, bulkhead, DLQ, degradation (5)
5 Grow	At 10x, what breaks first — reads, writes, storage, a hot key? Scales by adding machines or needs redesign?	Capacity plan	Horizontal scaling, LB, auto-scaling, pooling (6)
Then	Cross-service correctness? Entry point? Deploy & observe? Pipelines/AI?	Refinements	Chapters 7–11

Tips: announce the five questions as your agenda; use them as the checklist when stuck; **Q4 is the most skipped and draws the most follow-ups**.

Back-of-envelope numbers

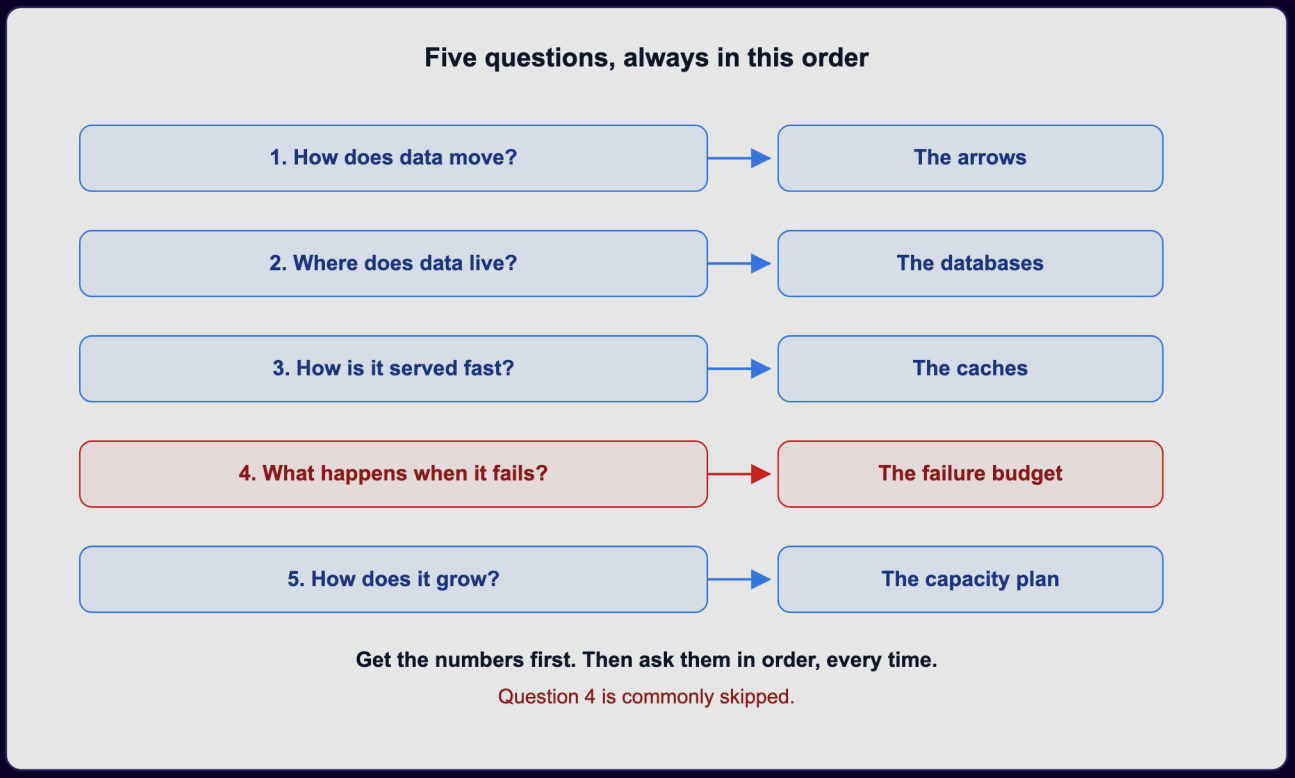
Latency	Time	Sizes / rules	Value
RAM read	~100 ns	Seconds per day	~100,000 (86,400)
Fast SSD read	~100 µs	Seconds per month	~3 million
Same-datacenter network hop (cache read)	~0.5–1 ms	Character	~1 byte
Simple DB query	~5–15 ms	ID	~16–36 bytes
Disk seek / slow query	~10–100 ms	Typical DB row	~1 KB
Cross-region round trip	~100 ms	Image	~100 KB–1 MB
"Feels instant" limit	~100–200 ms	1M × 1 KB / 1B × 1 KB	~1 GB / ~1 TB
Cache hit vs DB query	~10–15x faster	Peak	3–5x average

Availability	Downtime / year	/ month
99%	~3.7 days	~7 h
99.9%	~9 h	~43 min
99.99%	~53 min	~4 min
99.999%	~5 min	~26 s

The four calculations

#	Formula	Example
1	In flight = rate × duration (Little's law); drain time = backlog / drain rate	300 req/s × 2 s = 600 in flight (a 200-slot pool is too small); 552,000 msgs / 400/s ≈ 23 min
2	Chain availability = product (or add small failure rates)	0.999^5 ≈ 99.5% (5 × 0.1% ≈ 0.5% failure)
3	Avg latency = h·fast + (1−h)·(fast+slow); DB load = (1−h) × RPS	12,000 RPS, 1 ms hit, 15 ms DB: h=99% → ~1.15 ms, 120 QPS; h=90% → ~2.5 ms, 1,200 QPS
4	Per second = per day / 100,000; storage = records × size × retention	10M orders/day ≈ 100/s (peak 300–500/s); × 1 KB × 365 ≈ 3.7 TB/yr → one server, no sharding

Worked example — URL shortener (~60 s): 100M redirects/day → 1,000/s avg, ~4,000/s peak · 500M links × 1 KB ≈ 500 GB (one DB) · ~100:1 read:write; cache top 1% = 5M rows ≈ 5 GB; 99% hit → ~10 QPS to DB · 4,000/s × 2 ms = 8 in flight → one strong DB, one replica, a cache. Design: cache-aside with long TTL, replicas for misses, create path to primary (~10/s), click analytics on a queue off the redirect path, shard by short code later; growth risk = key generation (random codes need collision checks, counters need coordination).



In the interview

Q: How do you approach "design X"? **A:** Get the numbers, then walk five questions in order — how data moves, where it lives, how it's served fast, what happens when it fails, how it grows. Each surfaces problems that map to patterns; I check interactions and name each cost.

Q: Why not memorize known architectures? **A:** Requirements change; diagrams don't. Decomposing into known problems lets you rerun the same reasoning when a requirement moves (e.g., SMS via a rate-limited provider → webhooks + queue + idempotency + circuit breaker).

Q: How many concurrent requests at 300 req/s taking 2 s each? **A:** Little's law: 300 × 2 = 600 in flight, each holding a thread/connection.

Q: Five dependencies at 99.9% each — overall? **A:** ≈ 99.5%. Chains can only lower availability, so reduce *required* dependencies rather than perfecting each.

Q: Is a 90% hit ratio "almost" 99%? **A:** No — misses go from 1% to 10%, so the DB takes 10x the load. Judge caches by misses.

Q: 10M records/day at 1 KB for a year — shard? **A:** ≈ 3.7 TB/year fits one ordinary server; the estimate argues *against* sharding now.

Q: Why estimate at all? **A:** A 30-second order-of-magnitude answer shapes today's design; stating assumptions lets the interviewer correct them; numbers remove components you don't need yet.

Gotchas

- Naming a pattern without its cost → always say the price in the same breath.
- Skipping Q4 (failure) → walk *every arrow*: no answer / fails once / down for ten minutes / retry doubles an effect.
- Precise arithmetic → round hard (day = 100k s), round demand up and capacity down, check units at the end.
- 90% ≈ 99% → 10x the misses.
- Chasing more nines → each extra nine costs ~10x engineering; cut required dependencies instead.
- Over-engineering → "at this scale we don't need that yet" is a strong answer when the math backs it.

TL;DR

A pattern = problem + solution + cost. In interviews: numbers first, then Move → Live → Fast → Fail → Grow, naming every trade-off; four formulas (Little's law, chain availability, hit ratio, per-day/100k and storage) size everything.

2. Moving Data

The arrows: seven ways for data to travel between services, and what each decides about waiting, downtime and duplicates.

Must-know

- **Rule 1** – request-response is the **default**; the other six are for when waiting is pointless or impossible.
- **Rule 2** – **anything that retries can deliver twice** → consumers must be **idempotent**.
- **Job vs fact** – a *job* is work done once (→ queue); a *fact* is something that happened (→ publish). Confusing them is the most common mistake.
- **At-least-once** – never lost, may arrive more than once. **Push vs pull** – push costs open connections; pull costs requests.

Request-Response

- **Problem:** every arrow needs a default; teams err both ways – async when someone is waiting, or long unprotected sync chains.
- **How:** caller waits for result or error. Pool connections, run independent calls in parallel (latency = slowest, not sum), wrap each call in timeout + retry + circuit breaker.
- **Trade-offs:** + immediate answer, errors come back where you can handle them, simple tracing / – both sides must be up, availability multiplies down the chain, sequential latency adds up, each waiting caller holds a thread.
- **Real systems:** auth, pricing, payment authorization, interactive reads.
- **One-liner:** "Default for anything the caller's next step depends on; protected by timeout, retry, breaker; anything nobody waits on moves to a queue or event."

Message Queue

- **Problem:** a sync pipeline must process as fast as work arrives – a flash-sale spike becomes an outage despite enough total capacity.
- **How:** broker **persists before acking** the producer; workers **ack when finished**; no ack before the **visibility timeout** → redelivery (so at-least-once); **competing consumers** each get one message; **depth** (and oldest-message age) is the health/autoscaling signal.
- **Trade-offs:** + spikes become backlog, worker outages become a longer to-do list, independent scaling, free retries / – latency by design, duplicates (idempotency required), lost global ordering (partition by key for per-key order), "accepted ≠ done" gap, a broker + DLQ to run.
- **Real systems:** AWS SQS, RabbitMQ, Kafka consumer groups (really a replicated log), Celery/Sidekiq/BullMQ; GPU batch inference and embedding pipelines; LLM batch APIs.

- **One-liner:** "API validates, enqueues and confirms in ms; workers drain at a sustainable rate; at-least-once so consumers are idempotent and failures go to a monitored DLQ."

Publish-Subscribe

- **Problem:** producer delivering to each consumer itself → deploy per new team, slower fan-out, partial failures.
- **How:** publish each **fact once** to a topic; every **subscription** gets its own copy, backlog, retries and DLQ; subscribing is configuration; log-based brokers add **replay**.
- **Trade-offs:** + producer cost independent of consumer count, teams release independently / − schema becomes a **public API** (registry, additive-only, deprecation windows), duplicates × N subscriptions, invisible coupling (keep a subscriber catalog), no "all done" signal, **retention window** − a consumer further behind loses events forever.
- **Real systems:** SNS → SQS, Kafka consumer groups, Google Cloud Pub/Sub; Redis Pub/Sub is fire-and-forget (offline subscriber loses messages).
- **One-liner:** "Broadcast across services, compete within a service; publish once, per-team subscriptions, idempotent consumers, alert on backlog age before retention."

Event-Driven Architecture (event-carried state)

- **Problem:** thin notification events send every consumer back to the producer's API − coupling and load return, just async.
- **How:** put the facts **in the event**; each consumer keeps a **local view**, rebuildable by replay. One owner per fact; views are copies, never truth. Decide contents **field by field** (widely needed → in event; sensitive/rare → ID lookup).
- **Trade-offs:** + independence, producer load independent of consumers, views rebuildable / − copies everywhere, always eventually consistent, personal data spreads (deletion touches N+1 places), bigger public schema, harder debugging (correlation IDs, tracing), replay must not re-trigger side effects.
- **Real systems:** LinkedIn (Kafka origin), Uber trip events, Netflix viewing events, bank ledgers.
- **One-liner:** "The event's contents are the key decision; for widely consumed facts I carry state so consumers read locally and survive producer outages; exactly-current reads go to the owner."

Webhooks

- **Problem:** outside events arrive as POSTs under the *sender's* timeout/retry/disable rules; inline work → duplicates, disabled endpoint, silent gaps.
- **How:** verify **HMAC signature + timestamp** → persist raw event → **return 200 in ms** (200 = received, not processed) → queue → workers **dedupe on event ID**, fetch current state before deciding → **reconcile by polling** from a bookmark + silence alarm.
- **Trade-offs:** + near-real-time cross-company updates, sender retries for you / − you adopt each sender's contract, a public endpoint to secure, cross-company debugging, the polling loop stays forever.
- **Real systems:** Stripe (signed, event IDs, days of retries), GitHub (`X-Hub-Signature-256`), Slack (200 within 3 s), Twilio, Shopify.
- **One-liner:** "Webhooks for speed, polling for truth: verify, store, ack, queue, dedupe, reconcile."

Server-Sent Events (SSE)

- **Problem:** polling cost grows with audience; most responses say "nothing yet".
- **How:** one never-ending HTTP response per client; server pushes text events; `EventSource` auto-reconnects with **Last-Event-ID** against a **replay buffer**; async I/O holds tens of thousands of connections per server; jittered reconnects.
- **Trade-offs:** + plain HTTP (proxies/auth work), built-in client, cost tracks events / − one direction only, long-lived connections to operate (drain, least-connections LB, heartbeats), resume window not history (reset event → refetch), text only.
- **Real systems:** every LLM chat app streams tokens over SSE (time-to-first-token), CI logs, dashboards, notification feeds.
- **One-liner:** "One-way live updates → SSE; move to WebSockets only when the client must stream back."

Bidirectional Streaming (WebSocket / gRPC streams)

- **Problem:** with more than one server, "send to the room" is a routing problem — the socket was never the hard part.
- **How:** two tiers — **gateways** hold sockets only; a **pub/sub backplane** (topic per room) routes to gateways holding recipients; presence in a shared store; history DB is truth. Own protocol: sequence numbers, heartbeats, reconnect-and-resume, **capped send buffers** (coalesce ephemeral signals, disconnect with resync hint).
- **Trade-offs:** + full duplex, lowest interactive latency / − you own the protocol, stateful edge (draining, reconnect stampedes), HTTP middleware blind after upgrade, backpressure to design, backplane limits fan-out.
- **Real systems:** Slack, Discord gateways, Figma (WebSocket + CRDTs), Envoy xDS/Kubernetes watch (gRPC), realtime voice with barge-in.
- **One-liner:** "WebSocket faces users; gRPC streams face services; gateways are disposable, the history DB is the truth."

Key facts & numbers

Situation	Pattern
Caller can't continue without the answer	Request-response
Work can happen later; nobody waits	Message queue
Something happened; several services care	Pub/sub
Consumers need the data without calling back	Event-driven (state in event)
Tell a system at another company	Webhooks
Continuous one-way updates to clients	SSE
Both sides keep sending	WebSocket / gRPC bidi stream

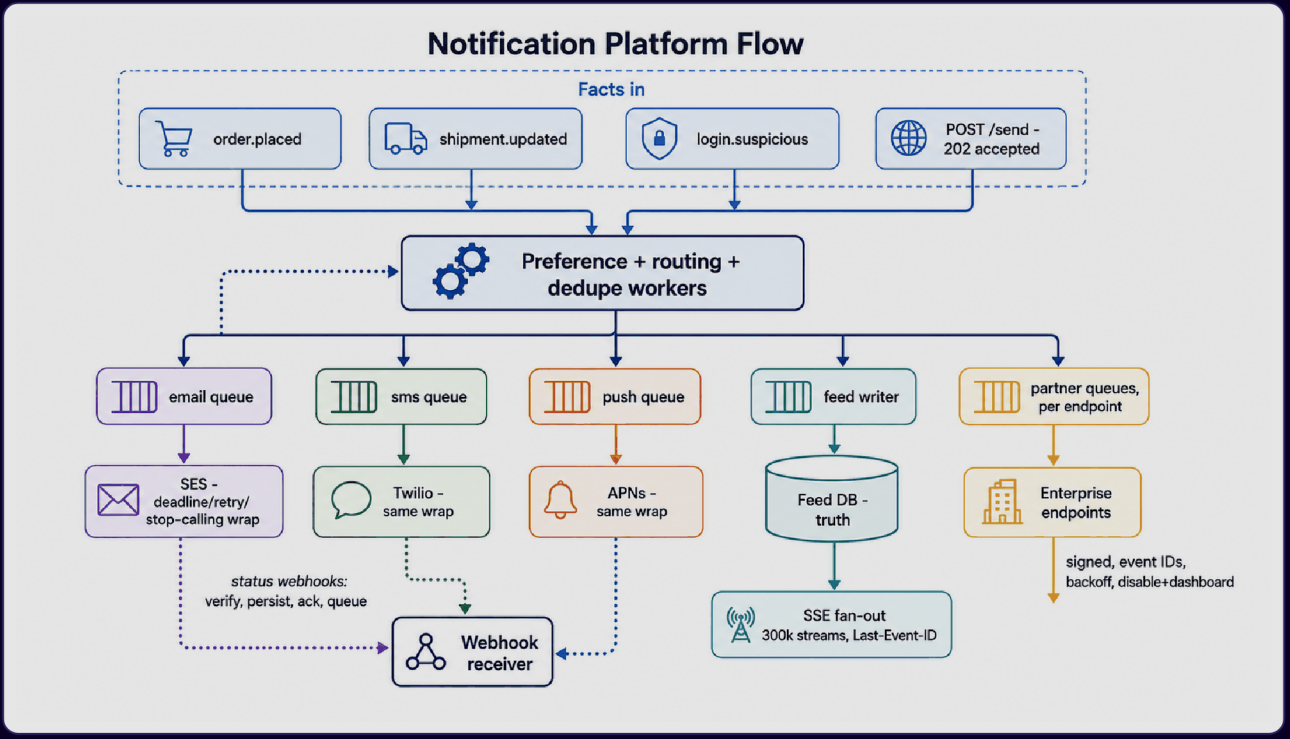
	Message queue	Pub/sub
Message is	A job	A fact
Delivered to	Exactly one worker	Every subscription
Adding a consumer	More workers for the same job	Another use of the same fact

	Choreography (pub/sub)	Orchestration (coordinator)
Control	Emerges	Explicit coordinator
"Is order X done?"	Nobody knows	Coordinator knows
Best for	Independent reactions (email, analytics)	Dependent steps (inventory → payment → shipping)

	Webhooks (push)	Polling (pull)
Freshness	Seconds	Poll interval
Completeness	Best effort	Guaranteed eventually
Worst failure	Silence (looks like a quiet day)	Stale but visible data

Push level	Use
Polling	Changes are rare
SSE	One-way server → client stream
WebSocket	Client streams back (chat, collaboration, games)
WebRTC	Media (audio/video)

Flash sale with a queue: 5,000/s in, 400/s workers → backlog ≈ 552,000 in 120 s → drain 552,000 / 400 ≈ 23 min; API still confirms in ~50 ms.



In the interview

Q: Why is this arrow synchronous? **A:** Because the caller's next step depends on it and a user is waiting (e.g., payment result); it's protected by timeout, bounded retries and a circuit breaker.

Q: Endpoint calls five services and is slow – fix without going async. **A:** Parallelize independent calls (sum → max), cache rarely changing answers, drop unneeded calls, reuse pooled connections.

Q: A worker crashes mid-message. **A:** Visibility timeout expires, broker redelivers to another worker, idempotency prevents a double effect; repeated failures move to the DLQ with an alert.

Q: Per-customer ordering on a queue? **A:** Partition by customer ID (Kafka partition key / SQS FIFO message group); order within a key, parallel across keys; a hot customer is capped at one partition.

Q: One subscriber is half as fast. **A:** Only its backlog grows; if it exceeds retention, events are lost – alert on backlog age and scale that subscription with competing consumers.

Q: Change an event schema without breaking unknown consumers? **A:** Additive optional fields only, schema registry in CI, v2 beside v1 for a deprecation window, consumer catalog.

Q: User exercises right to be deleted, but data is in a retained topic and six views. **A:** Delete at the owner, publish a tombstone; for retained history use crypto-shredding (per-user key, destroy it).

Q: How do you know you missed a webhook? **A:** You can't from the webhook path – reconciliation polling from a bookmark fills gaps; a silence alarm catches large ones.

Q: Why not WebSockets for everything? **A:** Upgrade, special infra handling and custom resume logic are wasted when the client only listens – SSE is plain HTTP.

Q: Alice on gateway A messages Bob on gateway B. **A:** A publishes to the room topic on the backplane; B (subscribed because it holds Bob) writes to Bob's socket; the message is persisted to history at publish time for resume.

Gotchas

- Queue/Kafka on a user-facing path (search) → keep what users wait for synchronous.
- Proposing a queue without duplicate handling → at-least-once means idempotent consumers.
- Several services on one shared queue called "broadcast" → each only gets a sample; use a topic with per-service subscriptions.

- Calling notification events "decoupled" while every consumer calls back → carry state, or accept the coupling.
- Business logic before returning 200 to a webhook → verify, store, ack, then process.
- Treating the SSE/WebSocket stream as storage → a durable DB is truth; clients refetch past the replay window.
- Unbounded per-connection buffers → cap, coalesce, disconnect with resync.
- Choreographing a dependent workflow → orchestrate it (saga).

TL;DR

Default to request-response; queue jobs, publish facts, carry state in events when many consumers need it, webhooks for cross-company push (with reconciliation), SSE for one-way streams, WebSockets only for true two-way — and assume every retried message arrives twice.

3. Storing Data

Where data lives as it grows: copies for reads, splits for writes, and logs for durability and history.

Must-know

- **Source of truth** — the one authoritative copy; everything else (replicas, views, caches) is derived.
- **Replication lag / read-your-writes** — replicas are slightly behind; a user may not see their own write.
- **Split brain** — two nodes both acting as primary; prevented by **fencing** the old primary.

Primary-Replica

- **Problem:** one machine is a read bottleneck (reads ~95% of traffic) and the only copy (SPOF).
- **How:** all writes → one primary, which streams its change log (WAL) to replicas; replicas serve reads and stand ready for promotion; fencing prevents split brain.
- **Trade-offs:** + read capacity scales out, failover in under a minute, safe place for backups/analytics / − lag and read-your-writes bugs, **zero extra write capacity**, failover only counts if practiced.
- **Real systems:** Postgres streaming / MySQL replication, MongoDB replica sets (elections), RDS/Aurora (up to 15 replicas, Multi-AZ).
- **One-liner:** "Scales reads and gives failover; route must-be-fresh reads to the primary; for write bottlenecks, shard — more replicas won't help."

Sharding

- **Problem:** writes and data size outgrow one machine; replicas can't help.
- **How:** split rows across shards by a **shard key** that is in nearly every query, spreads load evenly, keeps related rows together (commerce: customer ID); route by hash by default; each shard is its own primary-replica cluster.
- **Trade-offs:** + write capacity and size scale with machines, smaller blast radius / − scatter-gather cross-shard queries, cross-shard transactions in the app (saga), **hot keys** (hash spreads keys, not load), painful resharding, ops × shards (4 shards × 3 = 12 databases).
- **Real systems:** Instagram (user ID, shard in the ID), Slack (workspace), Vitess, Citus, MongoDB, DynamoDB/Cassandra partition keys.
- **One-liner:** "Last resort after bigger box, replicas, archiving and feature splits; the shard key choice, with reasons, is the answer."

Consistent Hashing

- **Problem:** `hash % N` bakes N into every address — going 4→5 shards moves ~80% of the data.
- **How:** servers and keys hashed onto a ring; key → next server clockwise; adding a server takes one slice (~1/N moves); **virtual nodes** (100+ per server) even out slices and spread a dead server's load.
- **Trade-offs:** + minimal movement, grow by one machine, no central directory / − vnodes mandatory, identical routing code everywhere, range queries scattered, storage handoffs still need ordering. Fixes *churn*, not *skew* (hot keys).

- **Real systems:** Akamai (1997), Dynamo (2007) → Cassandra, Riak, DynamoDB; memcached clients; Redis Cluster uses the hybrid (**16,384 hash slots** + directory); KV-cache-aware LLM routing.
- **One-liner:** "Resize moves ~1/N instead of ~everything; vnodes keep it even; for manual placement use hash slots plus a directory."

Write-Ahead Log (WAL)

- **Problem:** one logical write touches many scattered disk pages; power can fail between any two.
- **How:** append one checksummed record for the whole change → **fsync** → *then* acknowledge; apply to tables later; on restart replay from the last **checkpoint**, discard a torn tail by checksum; **group commit** covers many commits per fsync.
- **Trade-offs:** + "acknowledged = on disk", fast sequential writes, replication and **point-in-time recovery** almost free / − write amplification (written twice), fsync sets a latency floor, log housekeeping (stuck replica slots fill disks), recovery time bounded by checkpoint interval.
- **Real systems:** Postgres WAL (recovery + replication + PITR), InnoDB redo log, SQLite WAL mode, Kafka (the log as a product), Redis AOF + RDB.
- **One-liner:** "Log first, fsync, then ack; replay on restart; the same log gives replication and PITR."

Event Sourcing

- **Problem:** `UPDATE` destroys history − disputes, bug reach and future questions can't be answered.
- **How:** append immutable **business events** per entity (OrderPlaced, OrderCancelled) as the source of truth; state = replay (with **snapshots**); queries hit rebuildable **projections**; mistakes fixed by **compensating events**.
- **Trade-offs:** + audit by design, time-travel debugging, new analytics from old data, events double as outbox / − projections and lag, ~10–15× storage, schemas additive forever (upcast-on-read), awkward uniqueness, deletion via **crypto-shredding**.
- **Real systems:** double-entry ledgers, Git, Kafka-centric architectures, EventStoreDB/Axon/Marten; agent conversation traces.
- **One-liner:** "Only where history is the product (money, orders, inventory); for plain history, CDC is cheaper."

CQRS

- **Problem:** one model serves reads and writes badly − read indexes slow writes; reads still pay for joins/search.
- **How:** commands → lean normalized **write model**; queries → **read models** shaped per question (flat table, search index, aggregates), fed from one change stream (events or CDC) − **never dual writes**.
- **Trade-offs:** + each side tuned, per-model scaling, new reads = new projection / − eventual consistency (read-your-writes), a pipeline to run, duplicated data, over-used − needs a measured shape conflict.
- **Real systems:** e-commerce product pages, Twitter timelines (fan-out on write), feature stores, RAG (vector index = read model).
- **One-liner:** "Replicas fix how *many* reads; CQRS fixes what *shape* reads need."

Key facts & numbers

	Async replication	Sync replication
Ack to client	Immediately	After a replica confirms
Primary dies	Recent writes may be lost (~200 ms)	No confirmed write lost
Replica slow	Nobody notices	Writes stall
Fits	Feeds, catalogs, profiles	Money, inventory commits

Semi-sync (wait for exactly one replica) = common for money; choose mode per table/operation.

	Hash sharding	Range sharding
Spread	Even, automatic	Depends on ranges (time → hot current shard)
Range queries	Scattered	Excellent
Default	Yes (OLTP)	Time series, leaderboards

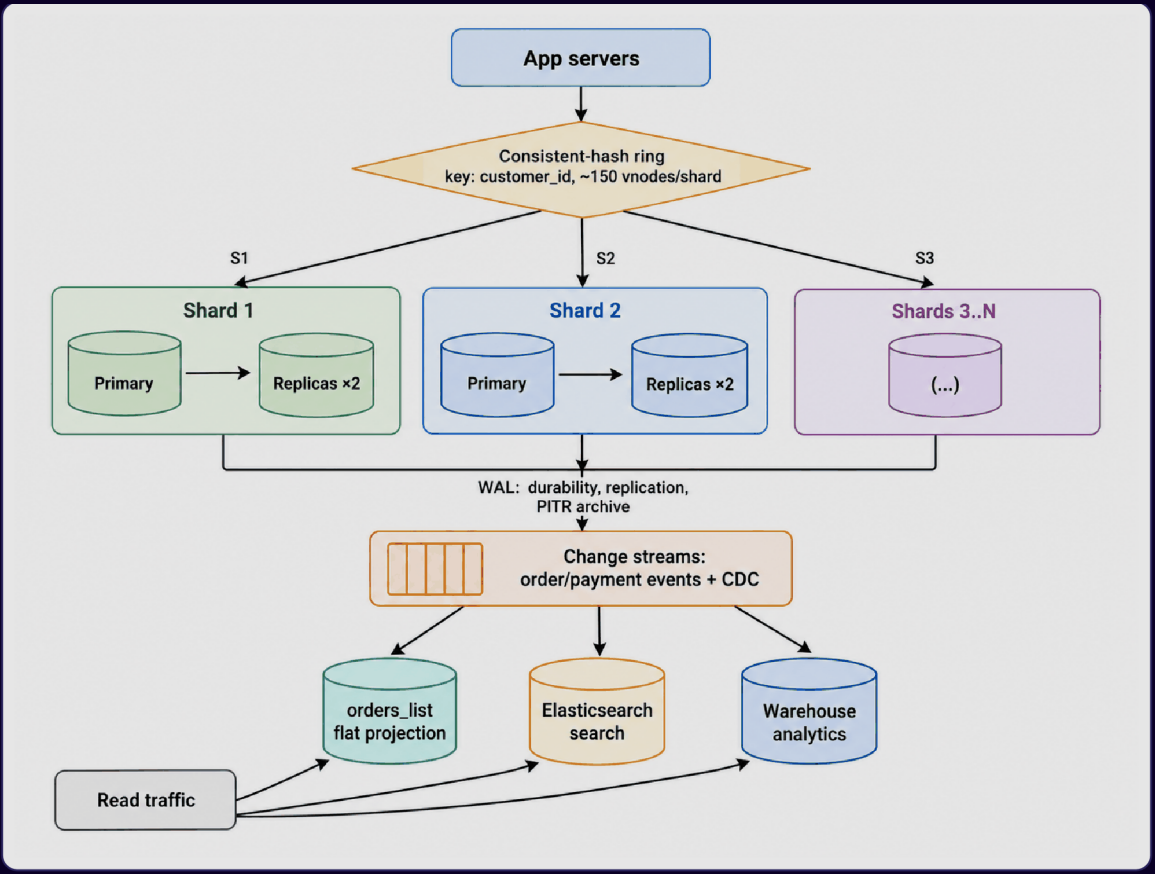
	Ring	Directory	Hybrid (hash slots)
Moved on resize	~1/N	Your choice	One directory row per slot
Placement control	None	Full	Per slot
Extra infra	None	HA metadata service	Small directory

	Snapshot (RDB)	Log (AOF/WAL)
Crash loses	Since last snapshot	~Nothing (unsynced tail)
Recovery	Fast	Replay (use snapshot + log)

	Event sourcing	CDC history
Truth	Event log	Current tables
Captures	Business intent	Row diffs
Adoption	Rebuild write path	Add a connector

	Read replicas	CQRS read models
Solves	Volume of reads	Shape of reads
300 ms six-way join	Still 300 ms	~8 ms (pre-joined)

Growth ladder (capstone, 100M users): replicas (reads) → shard by `customer_id` (writes) → consistent hashing (~150 vnodes/shard) for resharding → WAL + PITR (crash/corruption) → event-sourced orders (history) → CQRS read models (search, lists, warehouse).



In the interview

Q: The database is overloaded. **A:** First ask: by reads or by writes? Reads → replicas (and cache); writes/size → shard.

Q: A user's edit isn't visible after reload. **A:** Read-your-writes: route that user's reads to the primary for ~10 s after a write, or read only from replicas past their write's log position, or update the UI optimistically.

Q: How automatic should failover be? **A:** Detection and promotion automatic, but fencing first (split brain is worse than a dead primary), and practiced on purpose.

Q: Celebrity with 40M followers overloads a shard. **A:** Hash spreads keys, not load: cache the hot entity, isolate extreme keys on their own shards, or sub-shard that entity's data.

Q: Money transfer between users on different shards. **A:** Choose keys to avoid it; otherwise it's a saga, not a single DB transaction.

Q: Why virtual nodes? **A:** One point per server gives uneven slices and dumps a dead server's load on one neighbour; 100+ points even slices and spread failures.

Q: How does a DB survive a power cut without losing acked writes? **A:** WAL: append + fsync before ack; replay from checkpoint; torn tail discarded by checksum (it was never acked).

Q: An account has 2M events — replay all? **A:** No: snapshots every N events plus the tail, or a continuously updated projection.

Q: How do CQRS read models stay in sync? **A:** Consume one authoritative change stream (event log or CDC on the WAL), idempotently — never app-level dual writes.

Gotchas

- Read replicas for a write bottleneck → replicas add zero write capacity.
- "Failover is instant" → mention lag, data-loss window, split brain, practice.
- Sharding from day one without cost → last resort; justify the key.
- Modulo routing in an autoscaling design → consistent hashing / hash slots.
- "Consistent hashing fixes hot keys" → it fixes churn, not skew.
- "The DB writes it to disk" → explain the order: log, fsync, ack, apply.

- `OrderUpdated` events, event sourcing everywhere → business-intent events, only for ledgers.
- CQRS via dual writes, CQRS for a settings page, "CQRS needs event sourcing" → all wrong.

TL;DR

Replicas scale reads (not writes); shard on a well-argued key when writes/size outgrow one box; consistent hashing makes resharding move ~1/N; the WAL makes "saved" true and powers replication/PITR; event-source only ledgers; CQRS when read *shape* conflicts with the write model.

4. Caching

Serving reads fast: which caching pattern keeps each data type inside its staleness budget, and how to survive hot-key expiry.

Must-know

- **Hit ratio** – share of reads the cache answers; judge a cache by its **misses** (below ~90-something % it barely pays; at ~60% it adds a hop for little gain).
- **TTL** – time-to-live; always set one as insurance against write paths that forget to invalidate. **LRU** handles overflow.
- **Staleness budget** – how old each data type may be; decide per type, then pick the mechanism.
- **Invalidate = DEL, not SET** – concurrent writes can SET out of order and pin an old value until TTL; DEL costs one extra miss.

Cache-Aside (look-aside)

- **Problem:** the DB answers the same unchanged questions thousands of times a minute.
- **How:** read cache → on miss read DB and store with TTL; on write update DB then **delete** the key.
- **Trade-offs:** + ~100x less DB read load, 15 ms → 1 ms, full control per data type, cache death = slow but correct / – bounded staleness, invalidation chore on every present and future write path, boilerplate, misses cost double, cold starts and hot expiries.
- **Real systems:** Redis, Memcached, Facebook's memcache paper, `Rails.cache.fetch`, Django cache, Spring `@Cacheable`; CPU L1/L2/L3 and DB buffer pools.
- **One-liner:** "Check cache, fill on miss with TTL, DEL on write; TTL per data type; watch the hit ratio."

Read-Through

- **Problem:** cache-aside's miss logic is repeated in every function/service; forgotten steps and uncoordinated misses.
- **How:** app only calls the cache; on a miss the cache fetches via a configured **loader**, stores and returns – one implementation, can **coalesce** concurrent misses on a key into one DB query.
- **Trade-offs:** + one miss path, request coalescing, cleaner code / – loaders fetch one thing by key (computed/multi-source results stay cache-aside), needs a smart cache with DB access, coupling; writes still need invalidation.
- **Real systems:** loading caches, ORM caching layers, DAX-style caching services.
- **One-liner:** "Same read behaviour as cache-aside, different owner – the cache fetches, so misses are coalesced and implemented once."

Write-Through

- **Problem:** delete-on-write relies on every writer remembering, and costs a miss exactly when users read back what they saved.
- **How:** writes go through the cache: update DB synchronously, then the cache copy, then ack. If the cache update fails, **invalidate** and still return success.
- **Trade-offs:** + structurally fresh, read-after-write hits, pairs with read-through into one data layer / – extra write latency, churn from written-but-unread keys, guarantee ends at the cache boundary (bypassing writers still need TTL), smart infra.
- **One-liner:** "Cache never stale for writes that pass through it; ideal for edit-then-view."

Write-Behind (write-back)

- **Problem:** hot counters (views, likes) turn a DB row into a lock queue — e.g., 12,000 increments/s on one row.
- **How:** writes land in the cache and are acked instantly; dirty entries are **pinned** and flushed in **merged batches** every few seconds — DB load scales with distinct keys, not write rate.
- **Trade-offs:** + cache-speed acks, orders of magnitude fewer DB writes, absorbs bursts / − **durability window** (crash loses unflushed writes), truth inversion (DB readers see the past), heaviest machinery, harder debugging.
- **Real systems:** CPU write-back caches, social counters, Redis rate limiters/leaderboards, EhCache/Hazelcast.
- **One-liner:** "Fine for view counts with a documented loss window — never for money; if nobody reads the live value, use a durable queue + batching worker."

Cache Stampede Prevention

- **Problem:** a hot key expiring under load sends rate × rebuild-time identical queries to the DB (12,000/s × 50 ms = 600), feeding on itself; a cold cache does this for every key.
- **How:** **single flight / lock** (one rebuilder per key, lock has a short TTL); **serve stale** while one background refresh runs; **probabilistic early refresh** for the hottest keys; **TTL jitter**; cold-start runbook (warm top keys, ramp traffic).
- **Trade-offs:** + flat p99 through expiries, DB load follows distinct keys needing rebuild / − deliberate bounded staleness, lock bugs (use libraries), waiters held briefly, some wasted early refreshes.
- **Real systems:** Facebook memcache **leases**, `stale-while-revalidate` (RFC 5861), Go `singleflight`, XFetch paper; single flight for identical LLM prompts.
- **One-liner:** "Cap the crowd at one rebuild per key: lock, serve stale, refresh early, jitter, warm cold starts."

Key facts & numbers

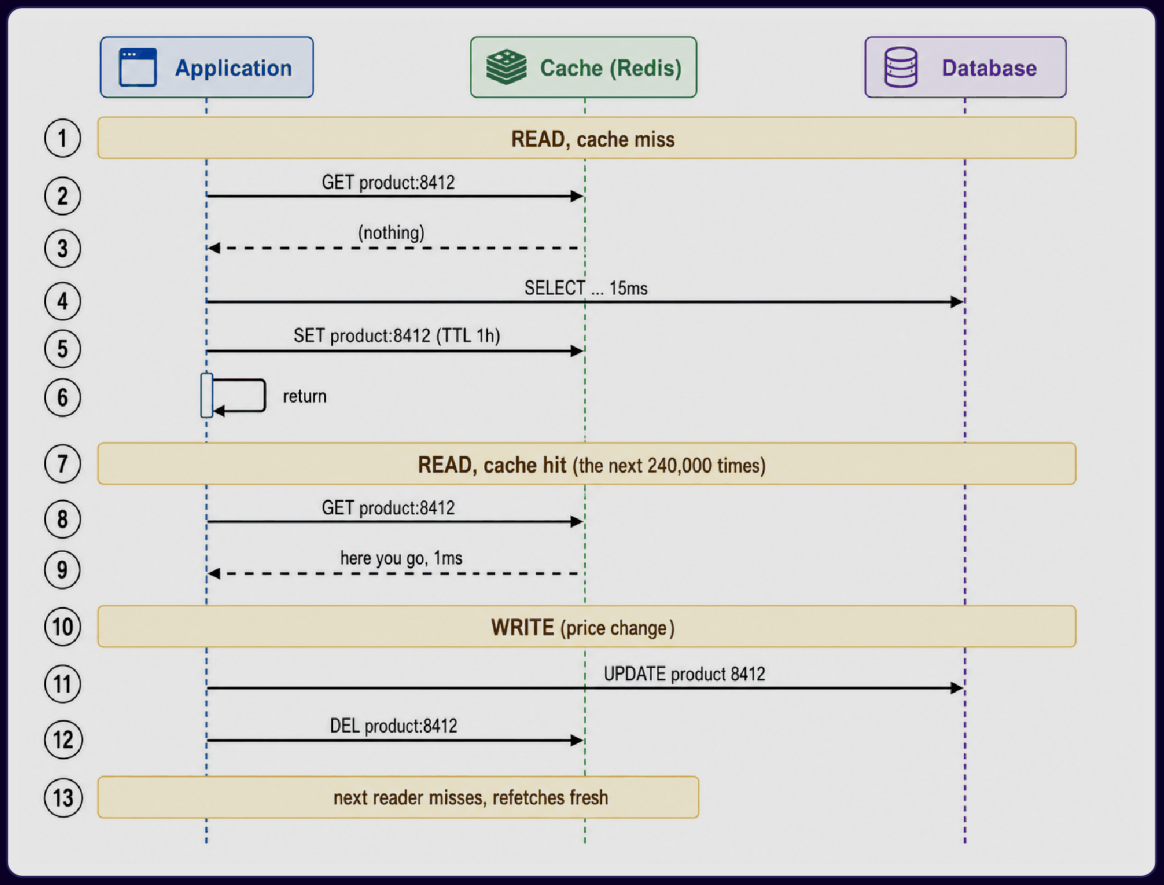
	DEL on write	SET on write
Next read	One extra miss	Immediate fresh hit
Race safety	Safe (reader fetches settled truth)	Risky (out-of-order SETs pin old value until TTL)
Remaining race	Slow reader re-inserts old value — bounded by TTL	–

	Cache-aside	Read-through	Write-through	Write-behind
Who fills	App on miss	Cache loader	Cache on write	Cache on write
Write ack after	DB write (+DEL)	(writes separate)	DB + cache	Cache only
Freshness	TTL/DEL bounded	TTL/DEL bounded	Fresh (for writes through it)	Cache is truth; DB lags
Risk	Forgotten DEL	Coupling	Write latency, churn	Loses flush window on crash

Stampede tool	Others get during rebuild	Staleness	Best for
Lock / single flight	A few ms wait	None	Staleness not allowed
Serve stale + refresh	Stale copy immediately	One rebuild's worth	Default for tolerant data
Probabilistic early refresh	Nothing (expiry rarely comes)	~None	Hottest keys at huge scale

Product-page data	Staleness budget	Pattern
Description	~1 h	Cache-aside, DEL on edit + 1 h TTL; serve-stale + single flight
Price (display)	Seconds	Cache-aside, DEL + 5 min TTL
Price on the order / inventory decision	Zero	No cache – read/conditional write on the source
Inventory badge	~10 s	TTL only + jitter
View counter	Live from cache	Write-behind, 2 s flush
Recommendations	Hours	Cache-aside, nightly invalidation, early refresh for top 100

Redis AOF `everysec` still allows ~1 s of loss.



In the interview

Q: Why delete instead of updating the cache on write? **A:** Two near-simultaneous writes can reach the cache in the opposite order to their DB commits, pinning the older value until TTL; DEL reduces the worst case to one extra miss.

Q: What TTL? **A:** Per data type: descriptions ~1 h, prices minutes (invalidation does the real work), inventory seconds or uncached – the purchase checks the DB.

Q: Cache-aside vs read-through? **A:** Same read result; cache-aside puts miss logic in the app (flexible, repeated), read-through in a loader (one implementation, coalesces hot-key misses; one-thing-by-key only).

Q: Product catalog read by forty microservices? **A:** Read-through as a shared caching layer – one loader, one TTL policy, coalescing.

Q: Write-through: DB write succeeds but cache update fails? **A:** Invalidate the key and return success; next read fetches the truth.

Q: Like button at scale? **A:** Diagnose row contention; write-behind – INCR in Redis, flush merged totals every ~2 s; accept and document the loss window.

Q: The write-behind cache node dies — what's lost? **A:** Up to one flush interval of dirty writes; shrink with a replica, AOF, or shorter intervals (moving toward write-through).

Q: Most popular key expires at peak? **A:** 12,000/s × 50 ms = 600 concurrent misses; single flight + serve stale + early refresh + TTL jitter.

Q: Whole cache cluster restarts empty at peak? **A:** Avoid cold restarts (replicas, consistent hashing); warm top keys from access logs, ramp traffic, single flight caps the DB at one query per distinct key.

Gotchas

- SET on write without knowing the race → DEL.
- Caching without both TTL *and* an invalidation plan.
- Caching a 900 ms query → look for the missing index first.
- "Read-through fixes invalidation" → it only controls reads.
- "Write-through means no TTLs" → bypassing writers exist.
- Write-behind for payments/orders; ignoring dirty-entry eviction; "Redis persistence = no loss".
- "On expiry it just goes to the DB" → simultaneous misses; client retries enlarge the crowd.
- Caching the inventory *decision* or the order price → always the source of truth.

TL;DR

Start with cache-aside (DEL on write, TTL per data type); read-through centralizes misses, write-through keeps written keys fresh, write-behind trades a loss window for counter throughput; cap hot-key rebuilds at one; never cache money decisions.

5. Reliability

Surviving calls that hang, fail, repeat or flood — seven patterns that each exist because the previous one shipped.

Must-know

- **A slow dependency is more dangerous than a dead one** — waiting holds threads, connections and locks.
- **Transient vs sustained** — retry the first, circuit-break the second.
- **Fail open vs fail closed** — degrade optional features; payment capture, auth and compliance fail closed.

Timeout

- **Problem:** calls with no deadline hang forever; 300 req/s × 30 s hang = 9,000 threads needed → service freezes in ~1 s.
- **How:** deadline per dependency at **2–3× its healthy p99** (p99 120 ms → 300–400 ms); connection timeout much tighter than request timeout; **deadline propagation** — each hop gets min(own limit, time remaining).
- **Trade-offs:** + bounded damage, hangs become countable failures (feeding retries/breakers), predictable worst case / − false failures (write uncertainty → idempotency), ongoing tuning, server work continues unless cancellation propagates (gRPC does).
- **Real systems:** gRPC deadlines; Python `requests` and raw TCP default to no real deadline.
- **One-liner:** "Every call has a measured deadline, and inner calls inherit only the remaining budget."

Retry with Exponential Backoff + Jitter

- **Problem:** brief failures are constant; instant retries synchronize into spikes against a struggling service.
- **How:** waits of ~100, 200, 400 ms…, **randomized** (jitter), capped attempts and delay, only **retryable errors** (timeouts, 5xx, 429 with `Retry-After`; not 400s), a service-wide **retry budget**, retry at **one layer**.
- **Trade-offs:** + transient failures disappear / − slower worst case, extra load in real outages, **3 × 3 × 3 = 27** requests if every layer retries, hides chronic failure (count retries), writes need idempotency first.
- **Real systems:** AWS SDKs ("Exponential Backoff and Jitter"), gRPC/Envoy retry budgets, Stripe (with idempotency keys), Kafka producers, Anthropic/OpenAI SDKs.

- **One-liner:** "Jittered exponential backoff, bounded, retryable errors only, one layer, idempotent writes — and a breaker when it's sustained."

Idempotency

- **Problem:** a timeout is uncertain — the write may have landed — so retries, redeliveries and double-clicks double-charge.
- **How:** client makes **one key per logical operation** before the first attempt and reuses it; server **atomically claims** it (insert-if-absent), executes once, stores the result, **replays** it for duplicates; states claimed → in progress → completed. Prefer naturally idempotent ops (set, conditional write).
- **Trade-offs:** + exactly-once *effect* on at-least-once delivery / − key store on the hot write path, client discipline, in-progress handling, side effects outside the boundary (emails, webhooks) need their own keys.
- **Real systems:** Stripe `Idempotency-Key` (responses replayed 24 h), Kafka idempotent producer (producer ID + sequence), SQS FIFO dedupe (5-min window), DynamoDB conditional writes, webhook consumers deduping on event ID.
- **One-liner:** "Exactly-once delivery doesn't exist; at-least-once plus an idempotent consumer is exactly-once processing."

Circuit Breaker

- **Problem:** a failing/slow dependency drains your threads through timeout-long waits; retries make it worse.
- **How:** three states — **closed** (count failures over a rolling window) → **open** (reject instantly after threshold) → **half-open** (a few test calls after cooldown) → closed on success. Breaker **outside** retries, counting final outcomes.
- **Trade-offs:** + isolation, fail in 1 ms not the full timeout, auto-recovery, relief for the dependency / − false trips (GC pause, DB failover), tuning (window, threshold, cooldown, probes; flapping), per-instance state, still needs a fallback.
- **Real systems:** Netflix Hystrix → Resilience4j, Envoy/Istio outlier detection, Polly, PyBreaker; per-provider breakers in LLM gateways.
- **One-liner:** "Retries assume transient, breakers assume sustained; every breaker needs an owner, a fallback and an alert."

Bulkhead

- **Problem:** shared pools let one feature/tenant legally take everything — with 0% errors (800 req/s × 0.6 s = 480 threads wanted, 200 available).
- **How:** walled pools (threads, connections, nodes) per feature/tenant/priority, sized by **rate × duration + headroom**; a full compartment rejects only its own overflow; alert on **per-compartment fullness**.
- **Trade-offs:** + protection when nothing fails, chosen blast radius, honest per-feature capacity / − idle capacity can't be lent, sizing drifts, more to monitor.
- **Real systems:** Hystrix thread pools, Kubernetes limits/node pools/quotas, Envoy per-upstream caps, PgBouncer per-service pools, separate GPU pools for interactive vs batch.
- **One-liner:** "Breakers watch failures, bulkheads watch occupancy — they stack."

Dead Letter Queue (DLQ)

- **Problem:** under at-least-once, a poison message is redelivered forever and blocks the line; dropping it is silent loss.
- **How:** after N attempts (with backoff), the broker moves it — with error details and attempt history — to a DLQ; **alarm at depth ≥ 1** for critical flows; runbook: inspect → fix → **redrive**.
- **Trade-offs:** + line keeps flowing, structured record, replay / − another queue to own (unwatched DLQ = loss on a retention timer), parked message breaks order, attempt counter can't tell poison from outage, PII in a second place.
- **Real systems:** SQS redrive policy, RabbitMQ dead letter exchanges, Kafka dead-letter topics (Connect, Spring), GCP Pub/Sub; corrupt PDFs in RAG ingestion.
- **One-liner:** "Park, alarm, fix, redrive — and for strictly ordered streams, pause the partition instead."

Graceful Degradation

- **Problem:** without fallbacks every optional widget is load-bearing; availability = everyone's numbers multiplied.
- **How:** rank features **with product**; per dependency a fallback ladder: fresh → cached → static default → hide → fail; alarm internally while degrading externally; exercise fallbacks (game days, chaos).
- **Trade-offs:** + availability independent of optional deps, gentler incidents / – fallback code rots unless run, owned staleness, masking hazard, many states to test.
- **Real systems:** Netflix popular titles when personalization fails (Chaos Monkey), Amazon protecting add-to-cart, Cloudflare Always Online, Facebook read-only mode, LLM gateway model → smaller model → semantic cache.
- **One-liner:** "Give up quality, not availability – except where a lesser answer is a breach or a double charge."

Key facts & numbers

Pattern	Watches / triggers on	Key number
Timeout	Elapsed time	2–3× healthy p99, per dependency
Retry	Retryable errors	~100/200/400 ms + jitter; ≤3 attempts; one layer
Idempotency	Duplicate key	Key created client-side before attempt 1
Circuit breaker	Failure rate (after retries)	e.g., 20-call window, 50%, 30 s cooldown, 5 probes
Bulkhead	Occupancy	Rate × duration + headroom per pool
DLQ	Attempt count	e.g., 5 attempts then park; alarm at depth ≥ 1
Degradation	Dependency unavailable	Ladder agreed with product

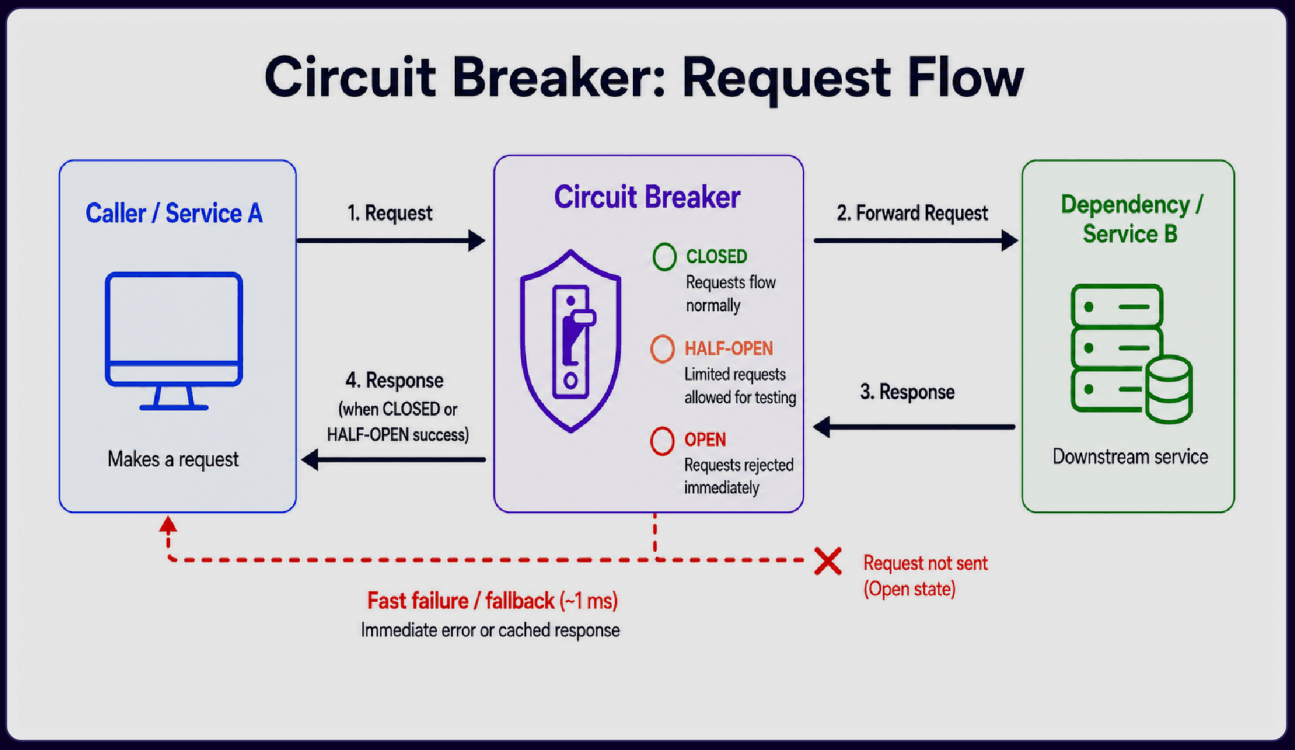
Situation	Reach for
Transient blip, answer needed now	Sync retry with backoff
Work needed, answer not needed now	Queue; consumer retries over hours; DLQ last
Sustained outage	Circuit breaker
Successful load hogging capacity	Bulkhead
Failures clearing in minutes–hours	Retry tiers (`retry-5m` → `retry-1h`) → DLQ
Deterministic bad payload	Straight to DLQ
Strict per-key order	Pause the stream, fix forward

You need	Build
No lost work	At-least-once delivery
No doubled effects	Idempotent consumer
"Exactly once"	Both

	Failover	Graceful degradation
Swaps	The dependency	The answer
Quality	Full	Lower on purpose
Order	First	When failover runs out

Payment-flow chain (capstone): timeout 1,200 ms (3× 400 ms p99), connect 200 ms, DB 250 ms → retry x3 jittered, one layer → idempotency key at Pay click → breaker (20/50%/30 s/5 probes; 1 ms rejects instead of 4

s) → bulkheads (checkout 100, refunds 40, status 40, reserve 20) → `order.paid` async + DLQ (5 attempts, alarm, runbook) → degradation (saved cart, authorize-later for trusted customers, capture fails closed).



In the interview

Q: What if this service just doesn't respond? **A:** Every call has a deadline from measured p99 × 2–3, tighter connect timeouts, and deadline propagation; the timeout becomes a countable failure for retries and breakers.

Q: Timeout fired but the write went through? **A:** Unavoidable uncertainty – make the write idempotent so the retry is safe.

Q: Could retries make an outage worse? **A:** Yes – backoff spaces them, a retry budget caps them, a breaker drops them to zero; retry at one layer only.

Q: Why jitter? **A:** Without it, clients that failed together retry together; backoff alone just reschedules the herd.

Q: Two requests with the same key at the same moment? **A:** Atomic insert-if-absent: one wins and runs; the other gets "in progress" then the saved result. Check-then-write is a race.

Q: Why can't the server generate the idempotency key? **A:** Only the client knows two requests are one logical operation.

Q: How do you pick breaker threshold and cooldown? **A:** Measure: window with 20+ calls, threshold well above normal error rate, cooldown ≈ typical recovery time; alert on every open.

Q: One endpoint gets 100x traffic – effect on others? **A:** Bulkheads per feature sized by rate × duration; the flooded compartment rejects its overflow; others unaffected.

Q: A malformed message is crashing the consumer. **A:** After N attempts it goes to the DLQ with its error; alarm; fix; redrive. Ordered streams: pause and fix forward.

Q: Which parts of your design are allowed to fail? **A:** Ranked with product: enhancements degrade via fallback ladders; auth and payment capture fail closed.

Q: How do you make a payment system reliable? **A:** Timeouts → jittered retries → idempotency keys → breaker → bulkheads → async + DLQ → degradation policy with fail-closed capture.

Gotchas

- "30 seconds" or library defaults → measured per-dependency values.
- A fresh full timeout at every hop → propagate the remaining budget (3 × 2 s = 6 s bug).
- "Retry 3 times" everywhere, on 400s, no jitter → one layer, retryable errors, budget.

- Retrying non-idempotent writes → key first.
- "Check if exists, then write" → atomic claim.
- "Exactly-once delivery" → at-least-once + idempotency.
- Circuit breaker for a hogging problem → bulkhead.
- DLQ without an owner/alarm; DLQ on ordered streams; DLQ for "payment declined".
- Degrading auth ("skip the check if slow") → fail closed.

TL;DR

Timeouts make hangs visible, jittered retries absorb blips, idempotency makes retries safe, breakers stop sustained storms, bulkheads contain success-shaped floods, DLQs park poison messages someone watches, and a product-agreed degradation policy decides what users see.

6. Scaling

How it grows: pick a direction, make servers interchangeable, spread and flex the load, and protect the database behind the fleet.

Must-know

- **Statelessness** — no server remembers anything between requests; "**cattle, not pets**". It's the prerequisite for horizontal scaling.
- **Scaling one tier multiplies load on the next** — capacity follows the bottleneck.
- **N+1** — enough servers for peak plus one spare for failure.

Vertical Scaling

- **Problem:** the machine is too small; teams skip the simple fix and buy years of distributed complexity.
- **How:** bigger machine — ~2 cores/8 GB up to 128+ cores/TBs of RAM (~**64×** on one setting); no new failure modes, single-machine guarantees intact.
- **Trade-offs:** + zero architecture change / — hard ceiling with a steep price curve near the top, no redundancy (bigger blast radius, still a SPOF), no elasticity, resize downtime; lock-bound or single-threaded software stops benefiting.
- **Real systems:** Stack Overflow on a handful of large machines; primary databases.
- **One-liner:** "Scale up to buy time; use the time to make the service stateless."

Horizontal Scaling

- **Problem:** one machine has a ceiling and is a SPOF; adding a second exposes hidden one-machine assumptions.
- **How:** statelessness checklist — sessions → Redis or signed tokens; files → object storage; in-process caches → shared cache (or short local TTL); crons → queue or distributed lock; counters/rate limits/locks → shared store. Then capacity = addition, availability = N+1, deploys = rolling.
- **Trade-offs:** + near-linear cost, N+1 availability, elastic / — the overhaul, ~1 ms per request for external state, shared stores become critical, multiplied downstream load (50 × 20 = 1,000 DB connections).
- **Real systems:** Twelve-Factor factor VI, Kubernetes ReplicaSets, AWS ASGs; inference fleets (weights load for minutes; KV cache affinity via consistent hashing).
- **One-liner:** "After 'add servers', list every form of state that lives on a server — that's the real work."

Load Balancing

- **Problem:** a naive round-robin splitter kept sending 1/6 of traffic to a server taking 20 s per request for 11 minutes.
- **How:** four jobs — **health checks** (active probes + passive error watching; removed in ~6 s), **distribution** (least-connections default; round-robin for uniform; weighted; hash for affinity), **draining** before replacement, **one stable address** (L4 by IP/port, L7 by path/header/cookie, TLS termination). HA via redundant pair + **floating IP**, or **anycast**.
- **Trade-offs:** + fast failure removal, zero-error deploys / — a critical hop, health checks wrong both ways (deep checks on shared deps pull the whole fleet), small settings with big blast radius.

- **Real systems:** nginx, HAProxy, AWS ALB (L7) / NLB (L4), Envoy, Cloudflare anycast; inference routers balance on queue depth and outstanding tokens.
- **One-liner:** "Round-robin assumes traffic is uniform; least-connections measures it. Health checks stay shallow."

Auto-Scaling

- **Problem:** fixed fleets pay for idle servers at 4 AM and drown when TV mentions you.
- **How:** control loop — measure signal (~1 min) → compare to ~**65%** target → **add fast, remove slowly** (cooldown, drain, hysteresis) → repeat. Signal: CPU for compute-bound; **requests per instance or p95 latency** for IO-bound; **backlog age** for queue workers. **Min = N+1, max = measured downstream capacity** (a bulkhead). Schedule known cycles, react to surprises, predict at large scale.
- **Trade-offs:** + ~half the compute spend, survives surprises / − **3–7 min reaction lag** (headroom bridges it), wrong signal disasters (CPU 15% while threads block), flapping, automated downstream flood without a max.
- **Real systems:** AWS ASG target tracking, Kubernetes HPA + cluster autoscaler, SQS-driven Lambda, Netflix Scryer (predictive); GPU warm pools.
- **One-liner:** "The signal is the decision; the max comes from what the database can survive."

Database Connection Pooling

- **Problem:** each connection costs a 20–50 ms handshake (TCP + TLS + auth) and a DB process's memory; per-server pools multiply (50 × 20 = 1,000; Postgres default limit ~100).
- **How:** layer 1 — warm app-side pool per server (10–20); layer 2 — shared pooler (**PgBouncer**, RDS Proxy) multiplexing to ~tens of real connections, sized near DB cores (HikariCP: ~cores × 2). Little's law: 400 q/s × 5 ms = **2** busy.
- **Trade-offs:** + handshake once per lifetime, built-in bulkhead/backpressure / − pool exhaustion as an outage mode (borrow timeout, wait-time alerts, leak detection), housekeeping, transaction-mode breaks session variables/temp tables/some prepared statements.
- **Real systems:** PgBouncer, RDS Proxy (for Lambda bursts), HikariCP; separate pools for primary and replicas.
- **One-liner:** "Small pools beat big ones; a shared pooler caps the total no matter how many servers the scaler adds."

Multi-Region and Cell-Based Architecture *(added lesson)*

- **Problem:** three AZs don't survive a regional failure, and nothing contains your own bad deploy/config or one noisy tenant — every blast radius is 100%.
- **How:** DR strategy from **RPO/RTO** — backup-restore → pilot light → warm standby → active-active; data sorted into **home-region** (most; async copy for DR), **read-mostly** (one writer, replicas everywhere) and **truly global** (consensus, pays cross-region RTT ~75–200 ms); **static stability** (survivors carry the load without scaling: 2 regions = 2× capacity, 3 regions = 1.5×). **Cells** = fixed-size full stacks per customer slice behind a tiny fail-static router (mapping table, not `% N`); deploy one cell at a time. **Shuffle sharding:** 8 workers, 2 per customer → 28 pairs, 1/28 full overlap.
- **Trade-offs:** + region survival, blast radius 1/N, local latency / − 1.5–2× capacity, data homes and conflict rules, N stacks to operate, router and global services as the new shared domain, failover works only if drilled.
- **Real systems:** AWS cell-based guidance, Route 53 shuffle sharding (2,048 name servers, 4 per domain), Slack AZ cells, Netflix active-active regions, Spanner.
- **One-liner:** "Regions for the location, cells for our own mistakes — and a second region that depends on the first is not a second region."

Key facts & numbers

Situation	Pattern
The machine isn't big enough	Vertical
No machine could ever be big enough	Horizontal
Traffic must be steered across servers	Load balancing
Demand rises and falls hourly	Auto-scaling
DB connections are costly/multiplying	Connection pooling

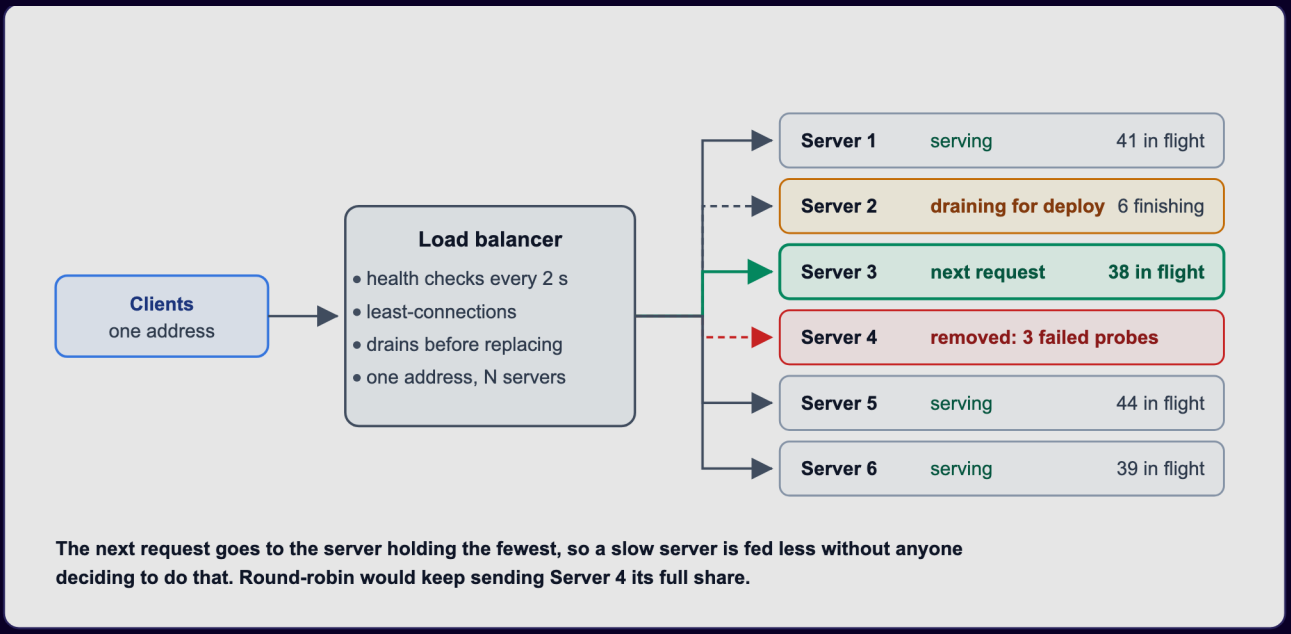
	Vertical	Horizontal
Ceiling	End of catalog (~64×)	None in app tier (moves downstream)
Cost curve	Steep at the top	~Linear
Availability	None added	N+1
Elastic	No	Yes
Prerequisite	None	Statelessness overhaul finished

	Round-robin	Least-connections
Slow server gets	Full equal share	Less, automatically
Mixed request cost	Imbalanced	Self-correcting
Long-lived connections	Bad	Right default

	Reactive	Scheduled	Predictive
Daily cycle	Late by the lag	Perfect	Good
Surprise spike	Yes (lag + headroom)	No	Only if it resembles history
Failure mode	Wrong signal, flapping	Wrong guess	Wrong with confidence

Capstone stage	Limit	Move	Deliberately not scaled
100 → 1k req/s	SPOF (40-min outage)	2+ servers + LB + statelessness (sticky sessions as a 2-week bridge)	Database
1k → 10k	DB reads (latency doubles)	13 servers, least-conn; cache-aside >98% hit (~40× fewer reads); replicas	No auto-scaler yet
10k → 100k	Idle 18 h/day + 3× spikes	Auto-scale (min 8, max 120), PgBouncer 2,400 → ~60 conns	CDN removes ~1/3 of requests
100k → 1M	Stateful tiers	~150–250k req/s reach 200–300 servers in 3 AZs; sharded DB, consistent-hash cache, replicated sessions	–

Sizing: 800 req/s per server, 4,000 peak → 5 needed → run 6 (N+1). Auto-scale: 65% of 800 ≈ 520 effective; 4 AM 900 req/s → 3 servers (N+1 floor); average fleet 8 → ~4.2.



The next request goes to the server holding the fewest, so a slow server is fed less without anyone deciding to do that. Round-robin would keep sending Server 4 its full share.

In the interview

Q: How would you scale this? **A:** Find the bottleneck; scale up to buy time; make the app stateless (sessions, files, caches, crons, counters) so identical servers behind a load balancer add capacity, give N+1, and roll deploys.

Q: One big machine or three small ones? **A:** Stateless service: three small (a failure costs a third). Primary DB or memory-locality workloads: one large.

Q: Doubled the fleet and latency got worse? **A:** The app tier wasn't the bottleneck — more servers multiplied DB connections/queries/contention; fix the data path.

Q: What should the health check test? **A:** Process, port, critical local resources — not shared dependencies (a Redis blip would eject the whole healthy fleet).

Q: Who balances the balancer? **A:** A redundant pair with a floating IP; anycast at internet scale.

Q: IO-bound API — scale on what? **A:** Requests per instance vs tested capacity, or p95 latency; CPU sits at ~15% while threads block on a slow DB.

Q: Scaler hit max and the site is still slow. **A:** Find the active bottleneck; if the DB is saturated, the max is working — improve caching/replicas/pooling and shed at the edge; raise the max only after measuring downstream capacity.

Q: App grew 10 → 50 servers; DB refuses connections. **A:** 50 × 20 = 1,000 connections; add a shared pooler (PgBouncer) and shrink per-server pools.

Q: Why does a bigger pool make it slower? **A:** DB runs ~one query per core; extra connections just queue inside it adding context switches and lock contention.

Q: Borrow timeouts after a deploy while the DB is idle? **A:** Connection leak (missing release in `finally`); leak detection + utilization/wait alerts.

Q: Survive a region outage? **A:** Pick backup-restore / pilot light / warm standby / active-active from RPO/RTO; home-region data with async DR copies; statically stable capacity (3 regions at 1.5× beats 2 at 2×); cells inside each region, deployed one at a time.

Gotchas

- Bigger machine after an *availability* incident → it's still one host.
- "Add servers" without checking state; sticky sessions as the permanent design.
- Every server running the cron → queue or distributed lock + idempotency.
- Round-robin with slow servers; restarting without draining; LB as if it can't fail.
- Deep health checks on shared dependencies.
- CPU as the universal scaling signal; no max; ignoring reaction lag.
- Up/down thresholds 5 points apart with no cooldown → flapping.

- "Make the pool 500" / raise DB limit to 5,000 → connections are not capacity.
- "Multi-region" with one shared global database, config store or sign-in service → the second region dies with the first.
- Growing a cell past its load-tested size → grow by adding cells.

TL;DR

Scale up to buy time, then out with stateless servers behind a least-connections, health-checked, draining load balancer; auto-scale on the right signal with max = downstream capacity; pool connections small, with a shared pooler – and always follow the moving bottleneck; past a million, cap the blast radius: statically stable regions and fixed-size cells deployed one at a time.

7. Consistency

How machines agree: all-or-nothing across machines, multi-step workflows, leaderless replicas, and detecting concurrent writes.

Must-know

- **CAP trade-off** – during a **network partition** you choose: stop serving (consistent, unavailable) or keep serving both sides and reconcile later (available, briefly inconsistent). Engineering can't remove it.
- **The cheapest distributed transaction is the one you never need** – shard-key discipline keeps a checkout on one shard; idempotent and **commutative** operations shrink what needs agreement.
- **Consensus (Raft, Paxos)** – needed when you want a single ordered winner; quorum counting alone doesn't give ordering.

Two-Phase Commit (2PC)

- **Problem:** one operation spans two machines; a crash in the gap half-commits it ("the $50 that vanished").
- **How:** **prepare** – each participant does the work, logs "ready" durably, keeps locks, votes yes (a binding promise); **commit** – if all yes, all commit, else all abort; the coordinator logs the decision.
- **Trade-offs:** + true atomicity, no visible half-state, looks like a normal transaction (XA/JTA) / – **blocking** (in-doubt participants hold locks while the coordinator is gone), two round trips + fsyncs, lock contention, availability multiplies down with participants, can't cross trust boundaries.
- **Real systems:** XA/JTA, Postgres `PREPARE TRANSACTION`, Google Spanner (2PC over Paxos-replicated groups), Kafka transactions, Flink transactional sinks.
- **One-liner:** "Real atomicity at the price of blocking – fine inside quorum-replicated databases, wrong between microservices."

Saga

- **Problem:** workflows across services can't use locks; a mid-flow failure leaves customers charged but nothing shipped.
- **How:** sequence of **local transactions**; progress written to a durable **saga log** before each step; on failure run **idempotent compensations** in reverse (refund, release); irreversible **pivot** step (shipping) goes last and recovers forward; an **orchestrator** owns the state machine; events published only from final states.
- **Trade-offs:** + no distributed locks, crosses companies, survives crashes, long waits cost nothing / – **no isolation** (visible in-between states, anomalies), undo code per step, workflow machinery, eventual consistency. Alert on workflows stuck past their SLO.
- **Real systems:** travel booking, Temporal/Cadence (durable execution), AWS Step Functions, order/payment "processing" states; agents with side-effecting tools.
- **One-liner:** "2PC hides the in-between state by holding locks; a saga shows it and manages it."

Quorum (leaderless replication)

- **Problem:** leaderless replicas accept writes but drift; reading a random one makes the cart flicker.
- **How:** N replicas, writes acked by **W**, reads ask **R**, with **W + R > N** so groups overlap; versions pick the newest; **read repair** + **anti-entropy** fix laggards; W/R tuned per operation. Standard: N=3, W=2, R=2.

- **Trade-offs:** + no leader, no failover pause, tunable per operation, tolerates stragglers / – concurrent writes split into siblings (need versioning), read/write amplification, fine print (**sloppy quorums**/hinted handoff, partial writes), single-key only – not a transaction.
- **Real systems:** Amazon Dynamo (the cart), Cassandra `ONE` / `QUORUM` / `ALL`, DynamoDB, Riak; etcd/ZooKeeper use consensus instead.
- **One-liner:** "Quorum promises visibility of completed writes, not ordering."

Vector Clocks

- **Problem:** concurrent writes split a key; timestamps "fix" it by silently dropping a real write – often the wrong one due to clock skew.
- **How:** one counter per writer (usually per replica), bumped each write; two versions either **descend** (replace) or are **concurrent** (keep both as **siblings**); the next reader merges by policy (union for carts, CRDT, ask the user) and writes back a version descending from all.
- **Trade-offs:** + nothing lost silently, immune to clock skew / – you must write/own merge policies, growth per writer, risky pruning, mental overhead.
- **Real systems:** Dynamo and Riak keep siblings; Cassandra chooses LWW by default for simplicity.
- **One-liner:** "Vector clocks detect conflicts; they don't resolve them. LWW is a data-loss policy you sign up for."

Key facts & numbers

Situation	Pattern
All parties commit or all back out	2PC
Many-step workflow, any step may fail	Saga
Copies agree without a leader	Quorum
Who saw which update, in what order	Vector clocks

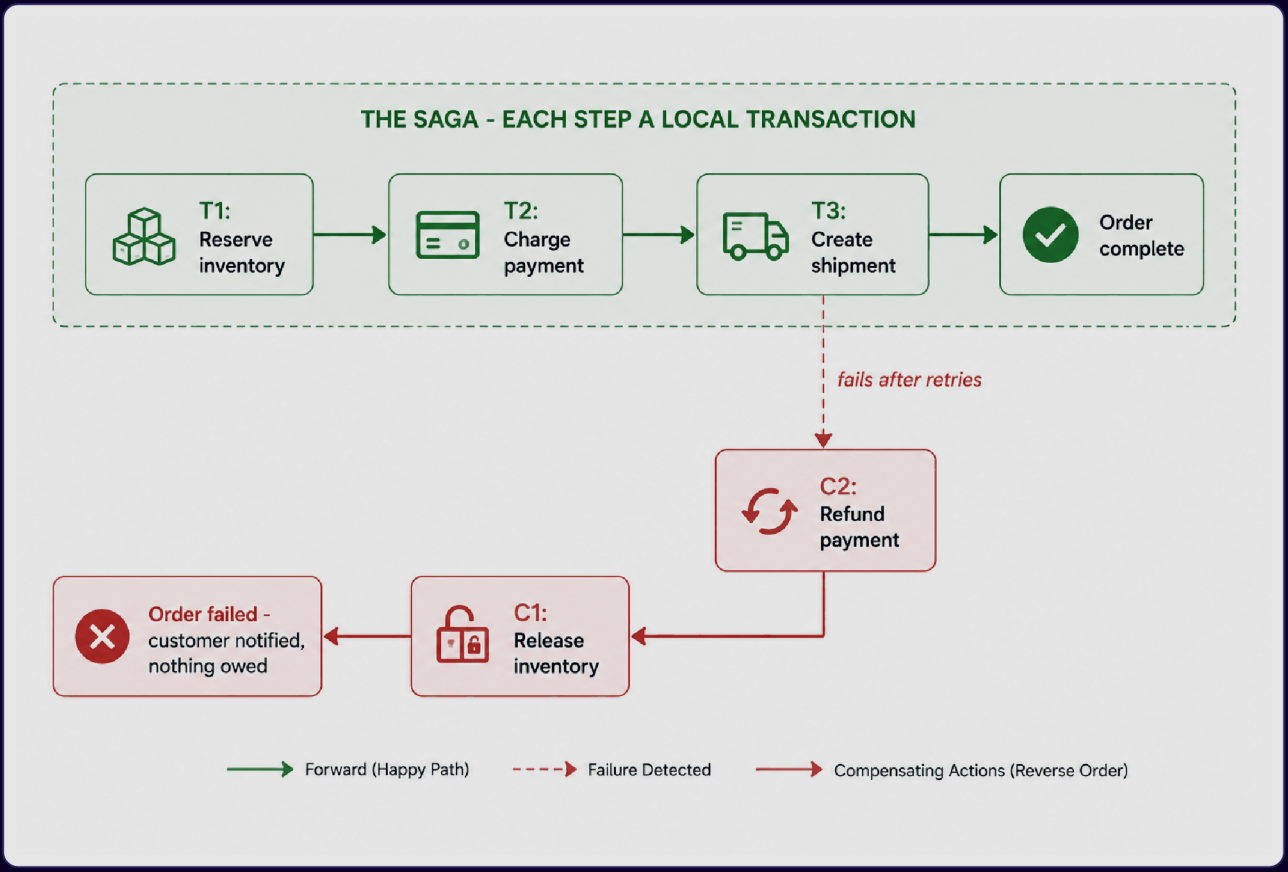
	2PC	Saga
Atomicity	Genuine (locks)	Imitated (compensations)
Intermediate states	Hidden	Visible
Coordinator dies	Participants frozen	Resumes from durable log
Fits	Short, internal, inside a DB	Business workflows across services
Fails by	**Freezing**	**Showing**

	Orchestration	Choreography
"Where is order 7f3a?"	One query	Hunt through N services' logs
Saga log	Explicit, resumable	Scattered, if any
Fits	Ordered, high-stakes workflows	Independent reactions

W, R on N=3	Behaviour
W=2, R=2	Overlap (4 > 3) – reads see completed writes; survives one replica down
W=3, R=1	Slow writes, fast fresh reads (config)
W=1, R=1	Fast, no overlap – eventual (fine for like counts)
W=2, R=1	3 is not > 3 → stale reads possible

	Primary-replica	Leaderless quorum
Write conflicts	Impossible	Possible (versions + resolution)
Failover	Seconds of write pause	None
p99	Leader's p99	Waits only for fastest W
Fits	Catalogs, orders, accounts	Carts, sessions, presence

	Last-write-wins	Vector clocks
Concurrent writes	One silently dropped	Both kept as siblings
Clock skew	Picks the wrong winner	Irrelevant
Burden	None	Merge policy per type



In the interview

Q: How can two services update their databases atomically? **A:** 2PC gives real atomicity but blocks on coordinator failure and can't include external parties; between services use a saga – local transactions, durable log, compensations, orchestrator.

Q: Coordinator crashes after both participants vote yes? **A:** Both are in doubt, holding locks until the coordinator recovers and replays its log (or an operator resolves them) – *blocking*.

Q: Why is 2PC fine inside Spanner? **A:** Participants are Paxos-replicated groups, transactions last ms, one trust boundary and one ops team – no single crash blocks it.

Q: Payment call timed out mid-saga – compensate? **A:** No – learn the true state first: retry with the same idempotency key or query the charge status.

Q: The refund itself fails? **A:** Compensations are retried idempotently with backoff; persistent failure goes to a dead-letter escalation queue for a human.

Q: N=3 – how many acks and read responses to see the newest write? **A:** W + R > N, e.g., W=2, R=2; versions pick the newest; read repair fixes laggards.

Q: Is W=2/R=2 strong consistency? **A:** No – overlap visibility plus conflict resolution; concurrent writes can split, sloppy quorums and partial writes weaken it.

Q: Two clients update the same key at once – who wins? **A:** Timestamps can't tell "overwrote" from "never saw"; vector clocks detect concurrency, keep siblings, and the reader merges by policy.

Q: Why not just synchronize clocks tightly? **A:** Writes 5 ms apart can still be concurrent; a later timestamp still discards a valid change.

Q: What's CAP? **A:** Under a partition you pick consistency (stop serving) or availability (serve and reconcile); each pattern makes that choice.

Gotchas

- 2PC for a multi-minute checkout or across companies → saga.
- "2PC is obsolete" → databases still use it internally.
- "Compensations" without saying what they are; orchestration state only in memory.
- Choreographing reserve → charge → ship → compensate via events → orchestrate.
- Quorum as a transaction across payment and inventory → single key only.
- Forgetting W + R > N; running on storage defaults unknowingly.
- "Vector clocks pick a winner" → they only detect concurrency.

TL;DR

Avoid distributed transactions via shard keys and idempotent/commutative ops; 2PC for short internal atomic work (it fails by freezing), sagas for cross-service workflows (they fail by showing); quorum W + R > N for leaderless visibility; vector clocks to detect – not resolve – concurrent writes.

8. API and Edge

The entry point: one proxy primitive in six roles, plus the contracts (limits, pagination, versions) that protect clients and capacity.

Must-know

- **North-south** – traffic crossing the boundary (clients, partners) → gateways, CDNs, rate limits, versioning. **East-west** – service-to-service → sidecars, mesh (mTLS, retries, tracing).
- **Proxy primitive** – reverse proxy + health checks + spreading = load balancer; + API policy = gateway; + geography + cache = CDN; per-instance = sidecar; + control plane = mesh.
- **AuthN at the entrance, AuthZ in services** – "can the rule be stated without domain words?" → gateway; otherwise service.

Reverse Proxy

- **Problem:** exposed app servers carry TLS, slow clients, visible topology and scattered policy.
- **How:** front server terminates TLS, **buffers slow clients** (threads = rate × processing time, not transfer time), serves static files, pools backend connections, forwards complete requests; one address, one enforcement point.
- **Trade-offs:** + absorbs internet messiness, cheap (well under 1 ms) / – an extra hop that must be redundant (pairs, anycast), header-trust discipline (**overwrite** `X-Forwarded-For`, trust it only from your proxy), not a security control by itself.
- **One-liner:** "Forward proxies hide who is asking; reverse proxies hide who is answering."

CDN

- **Problem:** distance – ~200 ms per ocean round trip, tripled by the TLS handshake – makes healthy origins feel broken far away.
- **How:** reverse proxy in ~200 cities with caches: **fingerprinted assets** (`app.3f9a2c.js`) cached forever/immutable; HTML with short `s-maxage` + `stale-while-revalidate`; TLS at the edge; **origin shield** merges misses; warm edge-to-origin connections for dynamic traffic; `stale-if-error`.

- **Trade-offs:** + 70–95% less origin load, DDoS absorption / – staleness across cities (purges propagate), cache headers are security-critical (private data in a shared cache), harder debugging (`x-cache`, `cf-cache-status`), per-GB cost, third party on the critical path.
- **Real systems:** Akamai, Cloudflare, Fastly, CloudFront, Netflix Open Connect (caches inside ISPs), Workers/Lambda@Edge.
- **One-liner:** "Fingerprint and cache assets forever; short TTL on HTML; purges only for emergencies; never cache personalized bodies."

API Gateway

- **Problem:** eleven services, eleven front doors – eleven auth implementations (one without revocation), topology as public API, policy drift.
- **How:** single north-south entrance: path routing (topology private), authenticate once and pass **verified identity** inward, limits/size caps/TLS, request ID for tracing. **Thin: policy yes, product no.**
- **Trade-offs:** + one policy point, private topology, root of every trace / – deliberate SPOF (redundant + thin), 1–2 ms per request, org bottleneck unless routing-as-code, constant pull toward a monolith/ESB.
- **Real systems:** Kong, Apigee, AWS API Gateway, Netflix Zuul, Envoy-based (Emissary, Gloo); LLM gateways.
- **One-liner:** "The gateway authenticates, services authorize; internal calls never detour through it."

Backend for Frontend (BFF)

- **Problem:** one uniform API fits every client badly – mobile pays six cellular round trips and 45 KB of unused data per screen.
- **How:** thin backend **per client experience**, owned by the client team, behind the gateway; fans out in parallel over datacenter links, returns one shaped response. No domain logic, no BFF-to-BFF calls, one per experience not per screen.
- **Trade-offs:** + per-client latency/payload, team independence, thin gateway / – N deployables, intentional duplication, business-rule leakage risk.
- **One-liner:** "BFFs shape and aggregate; GraphQL solves the same problem client-side with server-side cost."

Rate Limiting

- **Problem:** the entrance checks identity, not frequency – one buggy partner's 4,000/s loop takes everyone down.
- **How:** **token bucket** per trusted identity (API key) at the gateway: **r** = sustained rate, **b** = forgiven burst; cost-weighted endpoints; tiers per plan; shared atomic counters (Redis `INCR` + `EXPIRE`); refusal = **429** + `Retry-After` + budget headers.
- **Trade-offs:** + fairness by arithmetic, refusals cost ~0.1 ms vs ~50 ms to serve, capacity planning, pricing tiers / – small b punishes honest bursts, ~0.5 ms counter hop (or approximate local counts), NAT false positives with IP keys, many knobs, risky config changes.
- **Real systems:** Stripe, GitHub (`X-RateLimit-*`), Cloudflare edge limits, Envoy/nginx modules; LLM APIs limit tokens per minute.
- **One-liner:** "The rate limiter is the cheapest capacity I own."

Cursor-Based Pagination

- **Problem:** `OFFSET N` scans and discards N rows (linear cost), and positions in a moving list cause duplicates and skips.
- **How:** opaque token with the last row's sort values; next page = index seek `WHERE (created_at, id) < cursor` – constant cost, row-anchored; unique tiebreaker required.
- **Trade-offs:** + same cost at any depth, no drift, replica-friendly / – no jump-to-page-47 (seek by value instead), totals need separate COUNT, cursor lifecycle tied to sort order.
- **One-liner:** "Default for feeds, APIs, exports and infinite scroll; offsets only for shallow admin UIs."

API Versioning

- **Problem:** published APIs have clients you can't see or update – a rename broke 200 partners.
- **How:** **additive-first** (add, never rename/remove/retype; tolerant readers); coarse `/v2` via **gateway adapters** onto current internals (never a forked backend); lifecycle retirement – Sunset/Deprecation headers, per-client usage dashboard, outreach, **brownouts**, removal; **≤2 live versions**.

- **Trade-offs:** + change without breaking, fixable mistakes / – N versions to maintain, adapter rot, additive clutter, sunsets are organizational work.
- **Real systems:** Twitter v1→v2 (cautionary), 410 Gone brownouts.
- **One-liner:** "Additive first, versions coarse, and every version has a retirement date."

Sidecar

- **Problem:** east-west traffic needs mTLS/retries/timeouts/tracing across hundreds of entrances; per-language libraries drift.
- **How:** proxy process beside every instance over localhost; app speaks plain text locally; identical behaviour in every language; fleet-wide upgrades without app deploys.
- **Trade-offs:** + uniform behaviour, mTLS in one rollout / – ~50 MB per instance and ~1 ms per call multiplied across the fleet, startup ordering, own crashes, version skew, another debugging suspect.
- **One-liner:** "A sidecar is the building block; coordinating hundreds of them is the mesh."

Service Mesh

- **Problem:** hundreds of sidecars with no shared brain – certs expire on human schedules, configs drift, no place to write "who may call payments".
- **How:** **control plane** (watches registry, issues workload identities rotated hourly, holds declarative policy, streams config via **xDS**) + **data plane** (sidecars enforcing on every hop). Data plane **fails static** – control-plane outage freezes changes, not traffic.
- **Trade-offs:** + zero-trust mTLS, fleet-wide observability without instrumentation, canaries/mirroring as config / – a distributed system to run (platform team), sidecar costs, layered debugging, **global blast radius** for bad policy (e.g., a mesh-wide "retry 3x" duplicating non-idempotent order POSTs).
- **Real systems:** Istio, Linkerd, Envoy xDS, Istio ambient, Cilium (eBPF).
- **One-liner:** "Adopt only with dozens of services, several languages, a zero-trust mandate and a platform team."

Authorization Service *(added lesson)*

- **Problem:** 30 services with their own permission checks → BOLA leaks (`/invoices/8812` checked login, not ownership), removals that don't reach every service, slow recursive sharing, no audit answer.
- **How:** services are **enforcement points** calling `check(user, action, object)` on a central **decision point**. Models: **RBAC** (roles; role explosion), **ABAC** (attribute policies; OPA, Cedar), **ReBAC** (relationship graph). **Zanzibar tuples** `object#relation@subject` (`folder#viewer@team#member`); Check = short graph walk, Expand for audit, list-objects needs indexes. **Consistency tokens (zookies)** stop the "new enemy" seeing content created after removal. Cache sub-results, bulk-check pages, pre-filter search, fail closed, log every decision.
- **Trade-offs:** + one source of truth, audit, expressive sharing / – critical dependency on nearly every request, tuples to sync with domain data (CDC), list filtering is hard, policies need tests.
- **Real systems:** Google Zanzibar (>2 trillion ACL tuples, millions of checks/s, p95 < 10 ms and >99.999% availability over 3 years), SpiceDB, OpenFGA, Ory Keto, Airbnb Himeji, OPA, AWS Cedar.
- **One-liner:** "The gateway knows *who*; the service knows *which object*; the authorization service knows *whether*."

Key facts & numbers

	Fingerprinted URLs	Purge
Propagation	Instant (new URL)	Seconds–minutes
Stale possible	No	Yes, while spreading
Use for	Referenced assets	HTML, emergencies, takedowns

	API Gateway	Service Mesh
Direction	North-south	East-west
Identity	User/client (tokens, keys)	Workload (mTLS)
Policy	Auth, quotas, versioning	Retries, timeouts, mTLS, traffic splits
Blast radius	All incoming traffic	One service

	BFF	GraphQL
Shapes response	Client's server team	Client's query
Change needs	BFF deploy	Nothing (within schema)
Server cost	N thin services	Resolvers, N+1 queries, query cost limits
HTTP caching	Easy	Harder (POST)

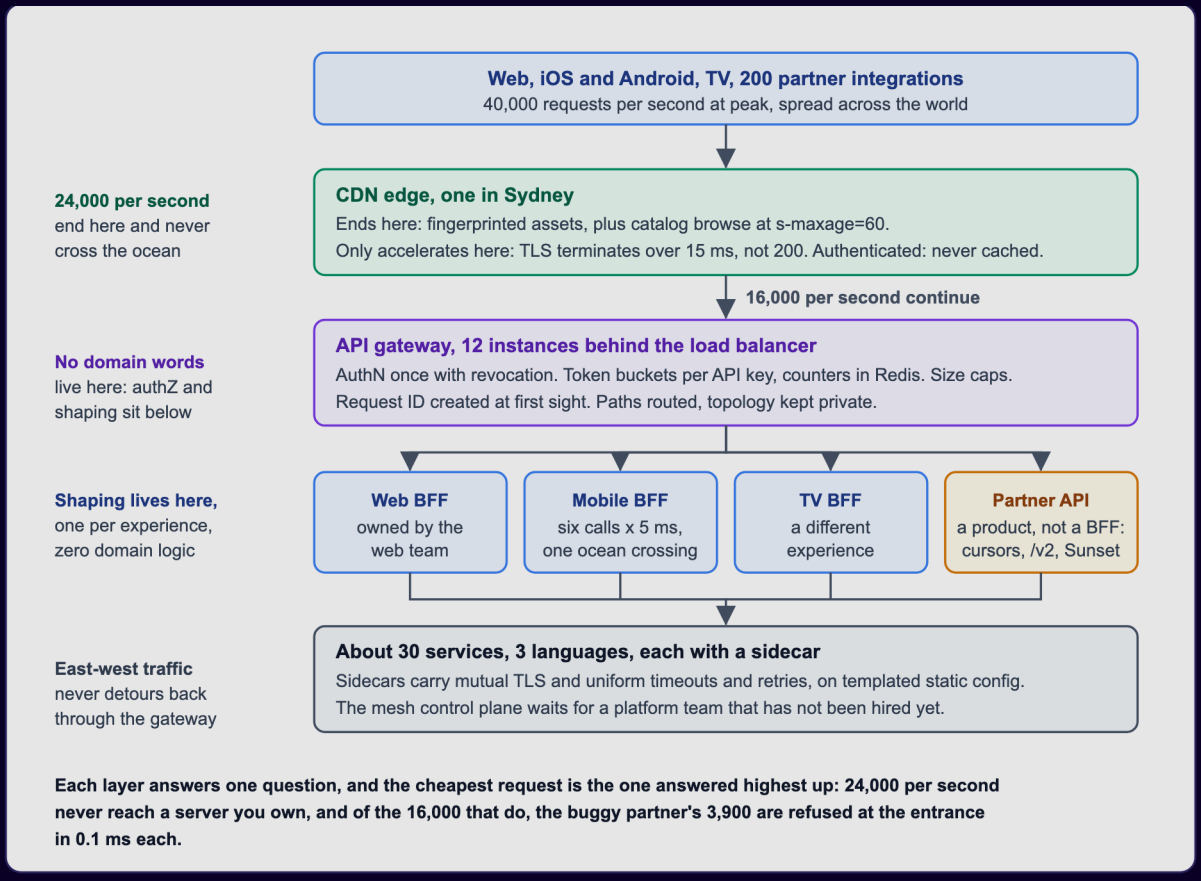
	Fixed window	Sliding window	Token bucket
Boundary burst	Yes (2× at edges)	No	No
Natural bursts	By accident	No	Yes, via b
Use	Rough protection	Near billing	Default

Three ways to turn work away: **rate limiting** (per identity, entrance) · **bulkhead** (per resource, inside) · **load shedding** (global emergency, low priority first).

	Offset	Cursor
Cost at depth	Linear	Constant
Changing list	Duplicates/skips	Unaffected
Page jumps / totals	Yes	No (seek by value)

	Shared library	Sidecar
Languages	One impl each, drifting	One binary
Upgrade	N teams' schedules	Fleet-wide in minutes
Cost	Zero latency, shared memory	~1 ms/call, ~50 MB/instance

Capstone: 40,000 req/s → CDN stops 24,000 (~60%) → 12-instance gateway → Web/Mobile/TV BFFs + Partner API (cursors, /v2, Sunset) → ~30 services in 3 languages with sidecars; buggy partner's 3,900/s refused at 0.1 ms each.



Code to remember

Cursor (keyset) pagination — the next page is an index seek, not a scan:

```
SELECT * FROM orders
WHERE (created_at, id) < (:cursor_ts, :cursor_id)   -- values decoded from the opaque token
ORDER BY created_at DESC, id DESC
LIMIT 50;
```

In the interview

Q: Why not expose app servers directly? **A:** A reverse proxy terminates TLS, buffers slow clients, serves static, pools connections, hides topology — and is where policy lives.

Q: Australian users see slow pages; Virginia servers are healthy. **A:** Distance, not capacity: CDN with fingerprinted forever-cached assets, short-TTL HTML with stale-while-revalidate, edge TLS, origin shield, warm connections.

Q: Banner must change now, but assets are cached forever. **A:** New image = new fingerprinted URL; only the short-TTL HTML changes (or one emergency purge).

Q: Can API responses go on the CDN? **A:** Public, identical-for-all responses yes (`s-maxage`, correct cache key); personalized bodies never.

Q: Isn't the gateway a SPOF? **A:** A deliberate shared dependency: redundant instances, health checks, anycast — and thin, so fewer changes break it.

Q: Can the BFF run checkout? **A:** It can start it and format status, but the money/inventory write is a domain-owned saga.

Q: Design a rate limiter. **A:** Token bucket per API key (r, b), cost weights, tiers, Redis atomic counters, 429 + Retry-After; IP limits only as an anonymous backstop.

Q: What's wrong with fixed windows? **A:** 100 at 11:59:59 + 100 at 12:00:01 = 200 in 2 s.

Q: 20 gateway instances — where do counts live? **A:** Shared Redis for exactness (sub-ms hop); local buckets with sync are approximate — OK for protection, not billing.

Q: Why isn't `created_at` enough for a cursor? **A:** Timestamps collide; use `(created_at, id)`.

Q: Change an API without breaking an un-updatable mobile app? **A:** Additive changes; coarse /v2 via adapters when unavoidable; retire with Sunset headers, dashboards, brownouts; ≤2 live versions.

Q: Mesh control plane goes down at peak? **A:** Sidecars keep last-known config (fail static); traffic continues; new workloads/certs/deploys blocked.

Q: How do 30 services enforce permissions consistently? **A:** Gateway authenticates and checks scopes; each service calls `check(user, action, object)` on a central authorization service (RBAC + ReBAC tuples + ABAC conditions), with consistency tokens, cached sub-results, fail-closed and decision logs.

Gotchas

- Trusting client-sent `X-Forwarded-For` → overwrite at the proxy, trust only from it.
- Tuning backend code for a geography problem; purge-only invalidation; caching personalized APIs.
- Business logic in the gateway; internal calls routed through it.
- BFFs with domain logic or calling each other; one backend per screen.
- Banning instead of slowing; IP as identity for authenticated traffic; silent 429s.
- Offsets for large feeds/partner APIs; timestamp-only cursors; parseable cursor format.
- Per-endpoint versions; versions without retirement dates.
- Mesh for five services; mesh-wide retries on non-idempotent POSTs.
- Checking "logged in" instead of "may access this object" (BOLA) → object-level check in every handler.
- Per-object permissions in JWTs → stale until expiry; keep tokens coarse.

TL;DR

Each request stops at the earliest layer that can handle it: CDN at the edge, a thin gateway for identity and token-bucket limits, BFFs for per-client shaping, cursors and additive versioning for durable contracts, sidecars/mesh for east-west trust — all one proxy primitive in different places. Object-level permissions are enforced in each service against one shared authorization service.

9. Operating in Production

Observe before you change, and make every change reversible: health checks, tracing, and three layers of progressive delivery.

Must-know

- **Golden signals** — traffic, errors, latency, saturation; canary verdicts come from these plus business metrics.
- **Three pillars** — **metrics ring the alarm, traces localize it, logs explain it**; put the trace ID in log lines.
- **Progressive delivery** — release in measured stages, observing each: blue-green (environment), canary (binary), flags (feature per user).
- **Expand-contract** — schema changes stay compatible with old and new code: additive first, dual-write, contract only after the rollback window closes.

Health Check Endpoint

- **Problem:** routing/restart machinery wants a verdict every ~2 s; shallow checks stay green while useless, deep checks turn the fleet red when a shared dependency blips.
- **How:** two endpoints — **liveness** ("am I broken in a way a restart fixes?", never dependencies → restart) and **readiness** ("should I get traffic now?", false while warming/draining, only truly essential deps → de-route). Both ~1 ms. Deep synthetic checks live in **central monitoring** → alerts, never routing.
- **Trade-offs:** + machine-speed removal, safe deploy ordering, self-healing / − ~10 lines whose mistakes hit the whole fleet (restart storms, fleet-wide de-routing, timeout-invented failures), constant probe load, false confidence.
- **One-liner:** "Liveness never checks dependencies; readiness includes only what the service is useless without."

Distributed Tracing

- **Problem:** a slow request's time hides *between* the logs of thirty services.
- **How:** gateway mints a **trace ID**; each operation records a timed **span** with parent and attributes; context (`traceparent`) propagates in headers and **queue metadata**; collector rebuilds waterfalls; **tail sampling** keeps errors + latency outliers + a small baseline.
- **Trade-offs:** + per-request evidence across services, visible retry storms/N+1/lock waits, real error origin, dependency map / − propagation discipline forever (one dropped header orphans the rest), sampling infra, instrumentation work, PII scrubbing.
- **Real systems:** Google Dapper, OpenTelemetry, Jaeger, Zipkin, Tempo, Honeycomb, Datadog; mesh skeleton; LangSmith for agents.
- **One-liner:** "Tail-sample errors and outliers; head sampling discards the interesting 1% before it knows."

Blue-Green Deployment

- **Problem:** in-place deploys make rollback a ~20-min redeploy; the new version meets real infra only once users are on it.
- **How:** two identical environments; deploy to idle **green**, verify with zero users, **flip the router** in seconds; **blue** drains and stays ready through the undo window; the shared DB never switches → expand-contract.
- **Trade-offs:** + rollback in seconds, whole-environment check, no mixed versions / − 2× capacity for the window (permanent on-prem), 100% exposure at the switch, tricky draining for long-lived connections, schema discipline forever.
- **One-liner:** "Switch in seconds, switch back in seconds – as long as the schema stays compatible for the whole rollback window."

Canary Deployment

- **Problem:** "it runs" ≠ "it behaves"; real traffic is the only complete test suite.
- **How:** **sticky, hashed 1%** cohort; compare against the **concurrent baseline** (not yesterday) on golden signals + business metrics + traces; automated gates **1 → 5 → 25 → 100**; failing score drains in seconds.
- **Trade-offs:** + blast radius as a dial, real-traffic verification / − hours per rollout, permanent mixed-version compatibility, statistics/telemetry rigor, version labels on all telemetry, tooling (mesh, Argo Rollouts, Flagger). Rare cohorts are invisible (0.4% of users × 1% = 0.004%).
- **Real systems:** Netflix Kayenta + Spinnaker, Argo Rollouts, Flagger, Google percent rollouts; model canaries through an LLM gateway.
- **One-liner:** "Canaries select traffic statistically; flags select people by name."

Feature Flags

- **Problem:** deploys do release's job – launch timing tied to release trains, killing one feature rolls back everyone.
- **How:** runtime switches evaluated **locally** from streamed, cached rules with **fail-static defaults**; deploy dark, release by flip/percentage/named cohort; kill one feature in seconds. Four types: **release** (temporary, delete at 100%), **ops kill-switch** (permanent, tested), **experiment** (stats window), **permission** (product config).
- **Trade-offs:** + deploy/release fully separated, per-feature kill switch, trunk-based dev / − **flag debt** (Knight Capital: $460M in 45 min from a reused flag name + dead code + partial deploy), 2^N states, config surface with global blast radius, runtime indirection (record flags as trace attributes).
- **Real systems:** LaunchDarkly, Unleash, OpenFeature, Facebook Gatekeeper, GitHub shipping dark; prompt/model versions as flags.
- **One-liner:** "Done means deleted; never reuse a flag name; never a network call per evaluation."

Containers and Kubernetes *(added lesson)*

- **Problem:** machines as the deploy unit – snowflake VMs, ~15% utilisation, 40-minute image bakes, humans restarting crashed processes.
- **How:** container = process + **namespaces** (what it sees) + **cgroups** (what it may use: memory over limit → OOMKilled, CPU over limit → throttled) + layered immutable **image** deployed by digest. Kubernetes = desired state + **control loops**: API server + etcd (Raft), scheduler, controllers, kubelet. Objects: Pod, Deployment (rolling via maxSurge/maxUnavailable, readiness-gated), Service

(ClusterIP/NodePort/LoadBalancer/headless), ConfigMap/Secret, StatefulSet, DaemonSet, Job/CronJob, HPA, PodDisruptionBudget. Patterns: sidecar, ambassador, adapter, init container.

- **Trade-offs:** + same artifact everywhere, density (440 VMs → ~24 nodes), self-healing / – a platform to operate, limit/probe traps, shared-kernel isolation, harder stateful workloads (operators).
- **Real systems:** Borg → Kubernetes, EKS/GKE/AKS, containerd; small teams: ECS/Fargate, Cloud Run, Container Apps.
- **One-liner:** "Declare the state, let control loops keep it true – and keep liveness probes off shared dependencies."

CI/CD Pipelines and Deployment Strategies *(added lesson)*

- **Problem:** monthly releases of ~400 commits, per-environment builds, click-made infrastructure, 6-hour hotfixes.
- **How:** trunk-based CI (commit stage < ~10 min); **build once**, promote one signed image by digest (+ SBOM); contract tests; progressive rollout with automatic verdicts and rollback. Strategies: **recreate** (downtime), **rolling** (waves), **blue-green** (switch; 2× briefly), **canary** (slice + verdicts), **shadow** (mirror, 0% exposure, stub side effects), **flags** (release by rule). Old and new versions coexist → expand-contract. **IaC** (Terraform plan/apply, locked remote state), **GitOps** (Argo CD/Flux pull desired state from Git), OIDC instead of long-lived CI keys. **DORA:** deploy frequency, lead time, change failure rate, time to restore.
- **Trade-offs:** + small batches, one trusted path, matching environments, speed *and* stability / – automation to build, compatibility discipline, a pipeline to secure, flaky tests to police.
- **Real systems:** Amazon (mean 11.6 s between production deploys, weekdays, May 2011), Spinnaker + Kayenta canary analysis, GitHub Actions/GitLab CI, Argo CD, Terraform.
- **One-liner:** "Build once, promote the same artifact, roll out progressively with an automatic undo."

Key facts & numbers

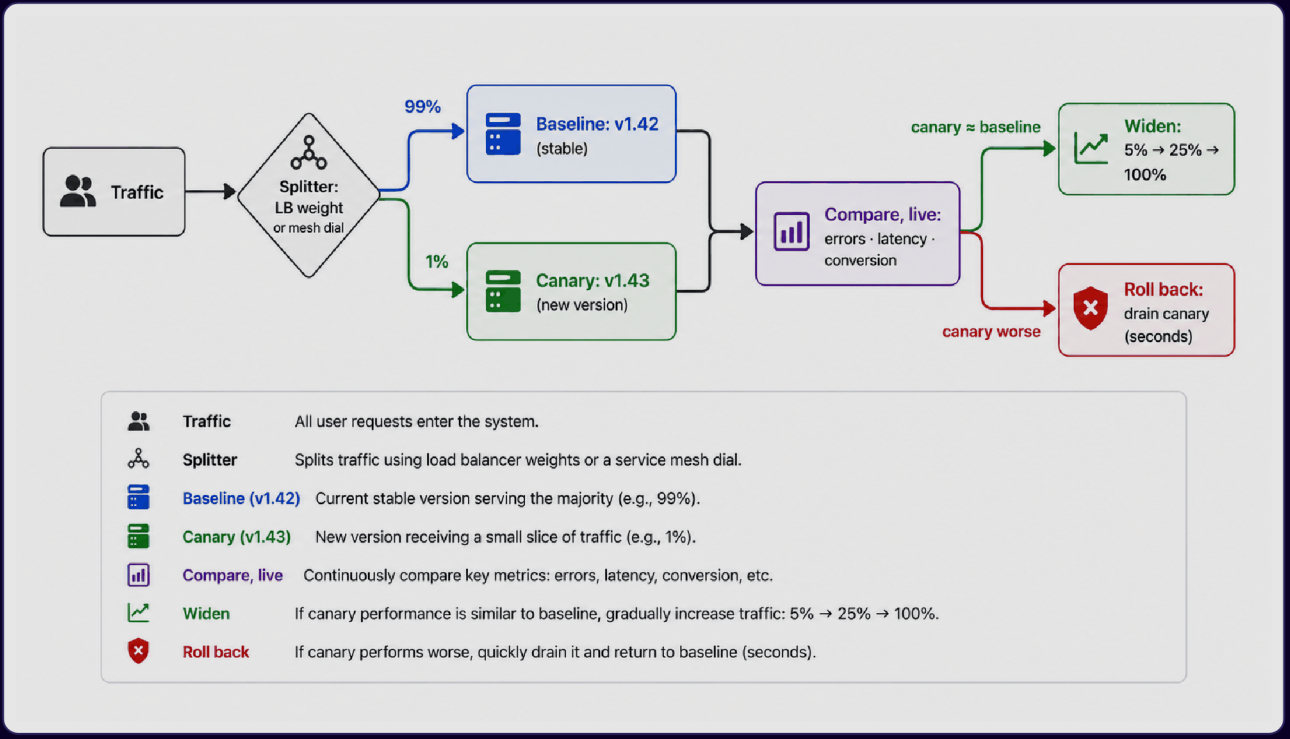
	Liveness	Readiness
Question	Am I broken?	Should traffic come to me?
Consequence	Restart	De-route
Dependencies	Never	Only essential, on purpose
Bad design causes	Restart storms	Whole fleet removed

	Metrics	Traces	Logs
Question	How much, how fast overall?	Which path did this request take?	What happened here exactly?
Cost	Small	Medium (sampled)	Largest
Role	Alerts	Localizes	Explains

	Rolling	Blue-green	Canary	Flags
Question	Cheap replace	Environment healthy?	Binary safe with real traffic?	Should this user see this feature?
Extra capacity	~1 instance	2× for window	~1%	None
Rollback	Redeploy (min)	Switch back (s)	Drain slice (s)	Flip (instant)
Version mixing	Yes, whole rollout	Never	Required	Per feature
Exposure	Gradual by instance	100% at switch	1→5→25→100	Named cohorts / %

Zero-downtime migration (undo column): 1 Expand (add nullable column – undo free) → 2 flagged dual-write + idempotent, throttled, checkpointed backfill (free) → 3 verify with comparison job + shadow reads → 4 switch

reads via flag ramp with canary verdicts (seconds) → 5a stop old writes (roll forward only, gated) → 5b drop column + flags (backup only). *Sequencing the undo options is the migration design.*



In the interview

Q: What does your health endpoint check? **A:** Liveness: process responsive, not deadlocked – no dependencies. Readiness: warmed up, not draining, plus only essential deps. Deep checks go to monitoring.

Q: DB failover of 90 s caused a 20-min outage with restarts everywhere. **A:** Liveness included a DB call → restart storm, cold instances, reconnect storms; liveness must check only the process.

Q: A request through thirty services is slow – find the delay. **A:** Distributed tracing: gateway-minted trace ID, spans per operation, propagated across HTTP and queues, waterfall shows where time hides.

Q: How does a trace cross a queue? **A:** Context in message metadata; consumer span linked to the producer span; the gap is queue wait.

Q: Zero-downtime deployment with instant rollback? **A:** Blue-green: verify idle green, flip router, keep blue warm; shared DB via expand-contract.

Q: You switched back to blue and it can't read recent orders. **A:** Green's migration broke compatibility; roll forward with a fix; prevent with expand-contract, contract after the window.

Q: Why compare canary to concurrent baseline, not a threshold? **A:** Errors/latency vary by time of day and mix; the concurrent 99% sees the same conditions.

Q: How long at 1%? **A:** Until the effect size you care about would have shown given cohort volume – minutes for big failures, hours for subtle ones.

Q: Flag service goes down? **A:** Nothing visible – local cached rules with declared defaults (fail static); only rule changes pause.

Q: How to stop flags piling up? **A:** Owner + expiry at creation, deletion in definition of done, reminders, lint against name reuse, monitor count and age.

Q: Walk me from commit to production. **A:** Trunk-based CI, one signed image built once and promoted by digest, IaC + GitOps for environments, canary with automatic verdicts and rollback; expand-contract because old and new run together.

Q: A pod restarts every few minutes, app log is clean. **A:** Check last state: OOMKilled (limit vs heap), failing liveness probe (probe checks a dependency?), or CrashLoopBackOff on start.

Gotchas

- Dependencies in liveness; one endpoint for routing *and* alerting; "ping the DB" in probes.
- "More logs" instead of tracing; 100% trace retention; propagation dying at the first queue.
- Rollback claimed safe after green wrote a format blue can't read.
- Canary vs historical data, manual dashboard watching, non-sticky routing, versions that can't coexist.
- Flags for authorization; network call per evaluation; no deletion plan; "a flag makes a destructive migration safe".
- Rebuilding the image per environment or deploying `latest` → build once, deploy by digest.
- Liveness probe that calls the database → restart storm on a blip.

TL;DR

Two cheap probes (liveness without deps, readiness with only essentials), tracing to localize, then change progressively – blue-green for the environment, sticky canaries against a live baseline for the binary, fail-static flags for features – with expand-contract keeping every undo cheap until the last, deliberate one-way door. All of it ships through one build-once CI/CD pipeline onto Kubernetes, whose readiness-gated rolling updates and control loops keep the fleet in its declared state.

10. Batch and Streaming

Data-intensive pipelines: computing over everything (batch), over the last few seconds (streaming), and keeping one truth, complete feeds, exact counts and bounded memory.

Must-know

- **Staleness budget chooses the engine** – hours → batch; one minute → micro-batch; one second → true streaming.
- **Event time vs processing time** – count when things happened, not when they arrived; processing time measures the network.
- **Watermark** – "no events older than T expected now"; tells a window when to close. **Late policy** is chosen per consumer.
- **Replayable log** – the backbone: rewind-based recovery, reprocessing, CDC feeds, seams between stages.

MapReduce

- **Problem:** one machine needs hours to read the data; at multi-hour runtimes, failure is routine.
- **How:** **map** (per record, where the data lives) → **shuffle** (regroup by key, all-to-all) → **reduce** (merge per key); **combiners** pre-aggregate locally; framework handles splitting, speculative copies for stragglers, re-execution (pure functions + immutable input).
- **Trade-offs:** + scale by addition, free fault tolerance, tiny model / – latency floor (minutes–hours, stale by design), rigid shape (disk between stages classically), shuffle-dominated cost, **skew**, a scanner not an index.
- **Real systems:** Google 2004 paper, Hadoop, Spark (in-memory DAGs, lineage, 10–100× faster iterative), BigQuery, Snowflake, dbt; training-data dedup; all-reduce in distributed training.
- **One-liner:** "The execution model outlived the implementation – nobody hand-writes MapReduce, but every engine still maps, shuffles and merges."

Stream Processing

- **Problem:** fraud/metrics/features "now" are about the last seconds of endless input; batch answers arrive hours late.
- **How:** a job that runs forever with **keyed state** over a replayable log; windows (**tumbling**, **sliding**, **session**); event time + watermarks; late policies per consumer; recovery = **checkpoint + rewind** (so sinks see replays).
- **Trade-offs:** + seconds-fresh answers, incremental cost, solid correctness machinery / – steepest concept stack, stateful ops (rescaling state is hard), provisional answers get amended, it's a service to operate.
- **Real systems:** Flink, Google Dataflow model (2015), Kafka Streams, Spark Structured Streaming (micro-batch); online feature stores.

- **One-liner:** "Every deploy rehearses the crash — so sinks must be idempotent or transactional."

Lambda and Kappa

- **Problem:** batch and streaming compute the same metric in two codebases that drift apart.
- **How:** **Lambda** — speed layer + batch layer + serving layer; the batch **overwrite** bounds any streaming error to one cycle. **Kappa** — one streaming codebase over a retained log; fix by **replay → verify → switch** (~50× real time).
- **Trade-offs:** Lambda: + automatic correction, recompute from raw / − every metric twice, drift, stitching seams. Kappa: + one codebase, reprocessing as routine / − retention cost (tiered storage helps), replay capacity, total dependence on the streaming engine.
- **Real systems:** Marz (Lambda) vs Kreps ("Questioning the Lambda Architecture"); Uber, Netflix, LinkedIn; Kafka tiered storage, Flink batch mode, Delta/Iceberg; feature stores fix training/serving skew.
- **One-liner:** "One definition, one logic source, one owner per metric — and never accidental Lambda."

Change Data Capture (CDC)

- **Problem:** derived systems (search, cache, warehouse) drift: dual-writes tear, polling misses deletes and intermediate states, bypass writes tell no one.
- **How:** tail the database's **WAL** like a replica; publish every committed insert/update/**delete tombstone** in commit order; snapshot to bootstrap; at-least-once to idempotent consumers; **outbox** table (same transaction) for intent-carrying domain events.
- **Trade-offs:** + cannot miss changes, one integration → N consumers, lag not loss / − table schema becomes a contract (registry/transform layer), **replication-slot retention can fill the primary's disk**, diffs without intent, another critical connector.
- **Real systems:** Debezium, Postgres logical replication slots, MySQL binlog, Fivetran, AWS DMS, Snowpipe Streaming, Netflix DBLog; re-embedding changed rows for RAG.
- **One-liner:** "Raw CDC for my own infrastructure; outbox for contracts with other teams; alert on slot lag."

Exactly-Once Semantics

- **Problem:** money-touching aggregates flip between double-counting (at-least-once rewinds) and silent loss (at-most-once); sink dedupe can't fix re-incremented state.
- **How:** at-least-once underneath + two couplings: **state ↔ log position** (checkpoint) and **outputs ↔ offsets** (Kafka transactions + read-committed, or deterministic **keyed upserts** — try first).
- **Trade-offs:** + money-grade correctness through crashes and deploys / − throughput tax, freshness tied to checkpoint cadence, subtle ops (barrier alignment stalls), boundary at external side effects, runs wrong code exactly once.
- **Real systems:** Kafka EOS (idempotent producer + transactions), Flink checkpoint barriers + 2PC sinks, Spark Structured Streaming, Google Dataflow; token-usage billing.
- **One-liner:** "Exactly-once delivery is impossible; exactly-once effect is at-least-once plus coupled state, offsets and outputs — inside the log ecosystem only."

Backpressure

- **Problem:** producers outrunning consumers is normal; an unbounded buffer schedules an out-of-memory crash.
- **How:** per stream choose **bounded buffer** (bursts), **deliberate drop** (freshness-only data: keep latest), or **slow the producer** (credits, windows, pull, 429). Logs between stages turn mismatch into visible lag.
- **Trade-offs:** + predictable memory, visible mismatch / − someone upstream feels it (design where), latency under pressure, slowest stage paces the pipeline and stalls checkpoints.
- **One-liner:** "Something has to give — buffer, drop or slow down; the only wrong answer is an unbounded buffer."

Partitioned Consumption

- **Problem:** parallel consumers break order; global order kills parallelism; keyed state needs each key in one place.
- **How:** hash key → partition (one ordered lane per key); each partition leased to **exactly one consumer** in the group; offsets per lane; partition count = parallelism ceiling chosen up front.
- **Trade-offs:** + per-key order *and* parallelism, locality for keyed state, poison blocks one lane / − repartitioning is surgery, rebalance pauses, hot partitions, order only within a key.

- **Real systems:** Kafka consumer groups vs SQS competing consumers.
- **One-liner:** "Order per key, parallelism across keys; check per-partition lag before total lag."

Inside Kafka *(added lesson)*

- **Problem:** slogans ("never loses messages", "RF 3 is safe", "exactly-once") hide the settings that decide the real guarantee.
- **How:** partition = folder of **segments** (`.log` + sparse `.index` + `.timeindex`, 1 GiB); retention deletes whole segments or **compacts** to latest per key; tiered storage to object storage (GA in 3.9). Fast: sequential I/O, page cache, **zero-copy** `sendfile` (lost under TLS), compressed batches end to end, long-poll pull. Replication: leader + fetching followers; **ISR** (caught up within 30 s); **high watermark** = what all ISR have = what consumers see; leader epochs truncate divergence; KRaft controllers (ZooKeeper gone in 4.0). Consumer groups: coordinator, `session.timeout.ms` 45 s vs `max.poll.interval.ms` 5 min, cooperative rebalancing, static membership, broker-side KIP-848 protocol (GA in 4.0, opt-in `group.protocol=consumer`). EOS: idempotent producer (PID + sequence), transactions (`transactional.id`, epoch fencing, commit markers, read_committed up to the last stable offset).
- **Trade-offs:** + throughput, durability without per-message fsync, replay / − guarantees are opt-in combinations, `acks=all` costs ms, a log not a smart queue.
- **Real systems:** LinkedIn (~7 trillion msgs/day, 2019), Confluent, MSK, Redpanda; Pulsar separates brokers from BookKeeper storage.
- **One-liner:** "RF 3 across zones, min ISR 2, `acks=all`, idempotence on, unclean election off, commit after processing, alert on lag age."

Key facts & numbers

	Classic MapReduce	Spark
Between stages	Disk	Memory
Shape	map → shuffle → reduce	Any DAG
Iterative work	Chain of jobs	10–100× faster
Fault tolerance	Rerun tasks	Rebuild from lineage

	True streaming	Micro-batch
Latency floor	ms	Batch interval (s–1 min)
Fits	Fraud, alerting, sub-second features	Dashboards, 1-min budgets
Example	Flink	Spark Structured Streaming

Window	Use
Tumbling	Revenue per calendar minute
Sliding	"More than 5 orders in any 10 minutes"
Session	Per-session metrics (gap-based)

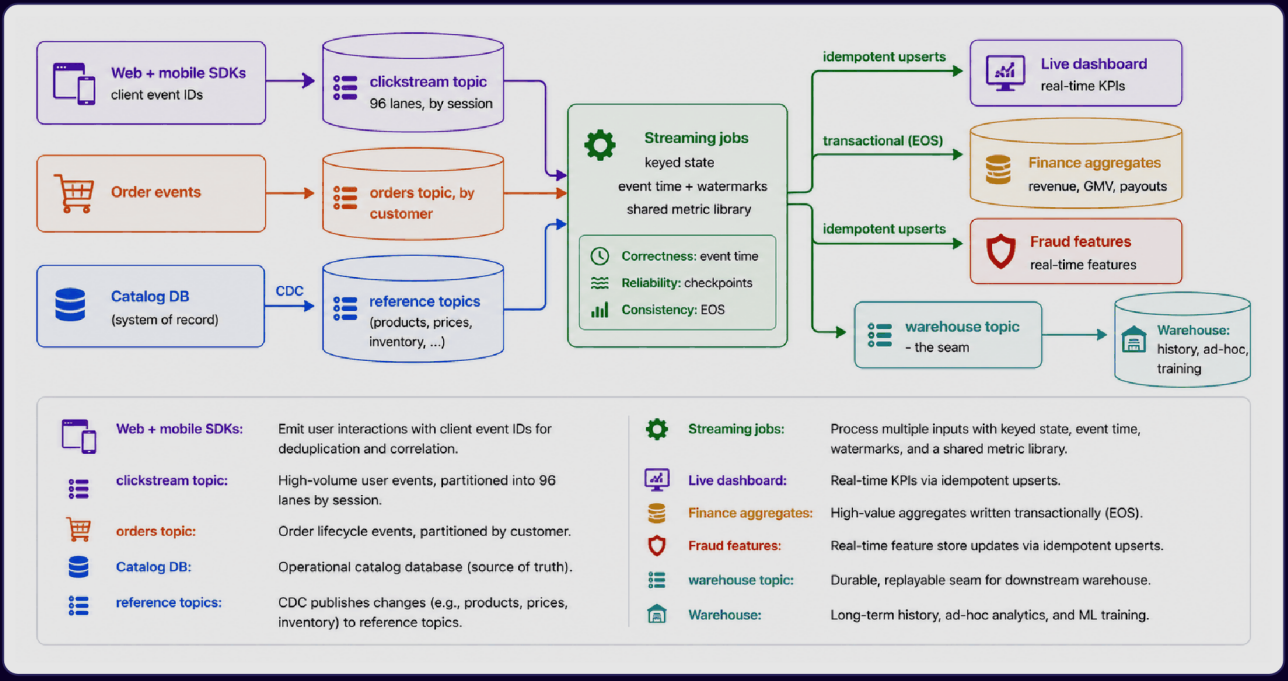
	Lambda	Kappa	Accidental dual path
Codebases	Two, on purpose	One	Two, by accident
Bug recovery	Next batch overwrites	Replay from offset	Manual backfill if noticed
Drift	By design	Impossible	Default

	Idempotent sink (keyed upsert)	Transactional sink
Freshness	Immediate	Checkpoint cadence
External stores	Any upsert-capable	Only inside the ecosystem
Use	Default (~80%)	Money, ledgers

Backpressure response	Fits
Bounded buffer	Short bursts
Drop (keep latest)	Positions, frames, sampled telemetry — never billing
Slow producer	Sustained mismatch

	Partitioned log	Competing consumers
Ordering	Per key	None
Scaling	By partition (fixed up front)	By worker, any time
Replay	Rewind offsets	Gone on ack
Poison message	Blocks its lane	Blocks nothing
Fits	Keyed facts	Independent jobs

Partition sizing: 50,000 events/s ÷ 2,000 per consumer = 25 minimum → choose 96–128 for years of growth. Skew fix: combiners, then **salting** (`electronics-1..16`) with a second merge stage.



Code to remember

Mergeable average (never average averages):

```
map(order)            -> (country, (amount, 1))
combine/reduce(pairs) -> (country, (sum_amount, sum_count))   # add pairwise
final                 -> sum_amount / sum_count
```

In the interview

Q: Count word frequencies over 10 TB. **A:** Map where the data lives → combiners → shuffle by key → reduce; reruns are safe (pure, immutable); shuffle is the cost; write it in Spark/SQL.

Q: 199 reducers finish in 2 min, one runs 6 hours. **A:** Skew from a hot key — combiners, then salt the key across reducers and merge.

Q: Real-time fraud detection? **A:** Keyed state by card, sliding event-time windows, watermark ~30 s, late events still update, checkpoint + rewind, sink deduped on (card, window).

Q: An event arrives 12 minutes late. **A:** Window already closed; per-consumer late policy — fraud updates and alerts late, dashboard drops, reconciliation side-channels.

Q: Streaming job was wrong for three hours. **A:** Kappa: replay from before the bug with fixed code, verify, switch. Lambda: next batch overwrites. Dual path: manual backfill.

Q: Keep search/cache/warehouse in sync with the DB. **A:** Log-based CDC (Debezium) with tombstones, idempotent consumers; outbox for domain events; alert on replication-slot lag.

Q: Why not dual-write? **A:** No transaction spans DB and index — a crash between writes causes silent permanent divergence.

Q: Can you guarantee exactly-once delivery? **A:** No; you can get an exactly-once effect by coupling state↔offset and outputs↔offset (transactions or keyed upserts). External effects need their own idempotency keys.

Q: Producer 5,000/s, consumer 3,000/s? **A:** Bounded buffer for bursts, drop for freshness-only data, slow the producer for sustained mismatch; a log between them makes it visible lag.

Q: Parallel processing with per-user order — how many partitions? **A:** Partition by user; one consumer per partition; count from throughput with years of headroom.

Q: Can Kafka lose messages? **A:** Yes, if configured to: `acks=0/1`, min ISR 1, unclean leader election, replicas in one zone, offsets committed before processing, or consumer lag beyond retention. Safe set: RF 3 across zones, min ISR 2, `acks=all`, idempotence, commit after processing, lag-age alerts.

Gotchas

- Batch for interactive/low-latency needs; ignoring the shuffle; averaging averages.
- Processing-time windows for money/fraud; per-event DB queries; state in app memory.
- Two pipelines without reconciliation (accidental Lambda); Kappa without retention math.
- Confident dual-writes; polling "complete" without deletes; raw CDC as the company event bus.
- "Exactly-once delivery exists"; transactions on a dashboard; sink dedupe for running aggregates; client retries with fresh event IDs.
- "We'd buffer it" (unbounded); dropping billing events.
- Assuming order across partitions; random keys when order matters; "add partitions later".
- "RF 3 = safe" without `acks=all` and min ISR 2.
- Kafka exactly-once assumed to cover DB writes or emails → idempotent sinks still needed.

TL;DR

Batch for completeness, streaming for freshness — over a replayable log with event time, watermarks and checkpoints; one metric definition across both; CDC for complete change feeds; exactly-once only where auditors look; bounded buffers everywhere; partitions for per-key order; and a Kafka configured for it: RF 3 across zones, min ISR 2, `acks=all`, idempotence, commit after processing.

11. AI-Era Systems

The same patterns inside ML and LLM systems, under three changed constants.

Must-know

- **Constant 1 — work is counted in tokens** (~3/4 of a word); one request can cost **100×** another, so anything counting *requests* (LB, rate limiter, capacity plan) is silently wrong.
- **Constant 2 — workers are costly and slow to boot**: a GPU machine ≈ 100× a web server; loading tens of GB of weights makes cold starts take *minutes*; the **KV cache** brings back session affinity.
- **Constant 3 — equality became similarity**: caches and indexes answer "what is near this?"
- **TTFT** — time to first token; streaming makes long generations feel fast.

- **Training/serving skew** — the model learns from one feature implementation and runs on another. **Leakage** — training rows that see the future.

Feature Store

- **Problem:** features coded twice (SQL for training, a service for serving) drift — great offline, silently failing live; each feature takes weeks.
- **How:** define each feature **once** in a registry; compile to an **offline path** (history + **point-in-time joins**, no leakage) and an **online path** (streaming into a millisecond Redis-class store); log-and-verify as drift alarm; backfill by replay; per-feature freshness tiers (streamed vs nightly at ~1/10 cost).
- **Trade-offs:** + skew removed by design, leakage prevented by the platform, shared features / − a platform to run, DSL limits (escape hatches reintroduce skew), freshness tiers, backfill compute.
- **Real systems:** Uber Michelangelo, Airbnb Zipline, Feast, Tecton, SageMaker/Vertex/Databricks; prompt "user context" is online feature serving.
- **One-liner:** "One author per feature, two execution paths — skew can't happen."

Model Serving (and Batching)

- **Problem:** one request at a time, LLM inference is memory-bandwidth-bound — GPUs ~8% busy; 20 req/s costs ~$5.6M/year.
- **How:** batch ~32 requests to share each weight stream (20–30× throughput for a few ms of TTFT); **continuous (token-level) batching** because output lengths vary 100×; **paged KV cache** (2–4× more concurrent requests); balance on queue length + pending tokens; sticky conversations; **buy** vLLM/TGI/Triton.
- **Trade-offs:** + 10–30× throughput per dollar / − small TTFT floor, shared fate (a giant prompt spikes p99 TTFT from 200 ms to 4+ s → chunked prefill, input tiering, prefill/decode disaggregation), complex engine, per-workload tuning.
- **Real systems:** vLLM, NVIDIA Triton, TGI, TensorRT-LLM, SGLang; provider batch APIs (~50% off).
- **One-liner:** "Same-sized jobs → batch by request; varying sizes → batch by token."

GPU Auto-Scaling

- **Problem:** minutes-long cold starts make reactive scaling late; idle GPUs cost ~$4/h; the real ceiling is provider quota.
- **How:** signal = **queue length + oldest waiting request age** (never CPU; GPU util misleads under continuous batching); **warm pool = spike rate × boot time**; calendar pre-warming; local weights + streamed images (~90 s boots); degradation ladder (queue → drop lower tiers → labeled smaller model); drain-then-park scale-down with an overnight floor; negotiated quota + spillover via the gateway.
- **Trade-offs:** + survives spikes, bill tracks demand / − paid idleness (warm pool), layered ops, preemption machinery, capacity procurement.
- **Real systems:** KEDA, Ray Serve, vLLM autoscaling, ASG warm pools and reservations, Modal/Baseten/RunPod, SkyPilot.
- **One-liner:** "Start API-first; own GPUs only when volume, privacy or custom models force it."

LLM Gateway

- **Problem:** 14 services calling providers directly — scattered keys (one leak = 14 deploys), unattributable spend ($40K runaway weekend), 14 homemade outage responses.
- **How:** one thin **egress** door: **key custody**, token metering with **hard budget caps** per team/feature, **logical model names** → providers (swaps are canaried config), per-provider timeouts/retries (Retry-After)/breakers feeding a **fallback ladder** (secondary provider → smaller self-hosted model → semantic cache → labeled error), guardrails once, **unbuffered streaming**, **no prompts**.
- **Trade-offs:** + policy once, failover built once, cost visibility / − critical-path dependency, streaming passthrough obligation, adapter churn (buy the core), scope creep toward a prompt CMS.
- **Real systems:** LiteLLM, OpenRouter, Portkey, Kong AI Gateway, Cloudflare AI Gateway.
- **One-liner:** "Module 8's API gateway turned outward — thin: no prompts."

Semantic Caching

- **Problem:** ~40% of questions repeat a meaning but only ~4% repeat the exact string.
- **How:** embed the query → nearest-neighbour search over cached entries → serve above a **conservative similarity threshold**; entity guards; never cache personal answers; topic tags + TTL invalidation; sampled

correctness audits; at the gateway behind an **exact-match tier**; **fail open**.

- **Trade-offs:** + 25–40% lower inference cost, faster latency class, stampede protection / – "confidently wrong" false hits, embedder + index on the hot path, fuzzy invalidation, keys tied to one embedding model.
- **Real systems:** GPTCache, Portkey, Cloudflare AI Gateway; provider **prompt caching** is different (reuses KV state for identical prefixes, 50–90% off input tokens, no false hits).
- **One-liner:** "Measure the repetition rate first – it's the whole business case."

Vector Database Sharding

- **Problem:** 500M embeddings need terabytes of RAM, and nearest-neighbour queries have no shard key.
- **How:** even shards + **scatter-gather** (local top-k, global merge, hedged requests, replicas); if every query has a filter (tenant), **shard on it** (single-shard routing, giant tenants get dedicated shards); one enormous corpus → **centroid routing (IVF)** probing 2–3 clusters (recall dial). Quantize first (int8/binary/PQ: 4–32× less memory). Re-embed by building a second index and switching.
- **Trade-offs:** + indexes beyond one machine, tenant isolation / – cost × shards, slowest-shard p99, measured recall SLO, score normalization, hot clusters/giant tenants.
- **One-liner:** "Compress before you add machines; shard on the filter if there is one."

RAG Pipeline

- **Problem:** knowledge in weights can't be updated, cited or access-controlled; fine-tuning fixes none quickly.
- **How: ingestion** – CDC on content sources → structure-aware chunking → embedding → sharded, ACL-tagged index (only changed pieces re-embedded); **query** – semantic cache → embed → top-k **filtered by the requester's ACLs** → cross-encoder **rerank** → cited generation via the gateway. Evaluate **recall@k** and **faithfulness** separately; debug retrieval first.
- **Trade-offs:** + updatable knowledge in seconds, citations, real access control, smaller models / – retrieval caps quality, two pipelines to run, hallucination reduced not removed, two-stage evals.
- **Real systems:** enterprise assistants (Glean-class), code assistants, agentic retrieval.
- **One-liner:** "RAG for knowledge, fine-tuning for behaviour, long context for small stable corpora."

Key facts & numbers

	Dynamic (request-level) batching	Continuous (token-level) batching
Membership	Fixed at start; all wait for the last	Join/leave every token step
100× output variance	Convoy effect	Handled
Fits	Embeddings, vision, rankers (Triton)	LLM generation (vLLM, TGI, TensorRT-LLM, SGLang)

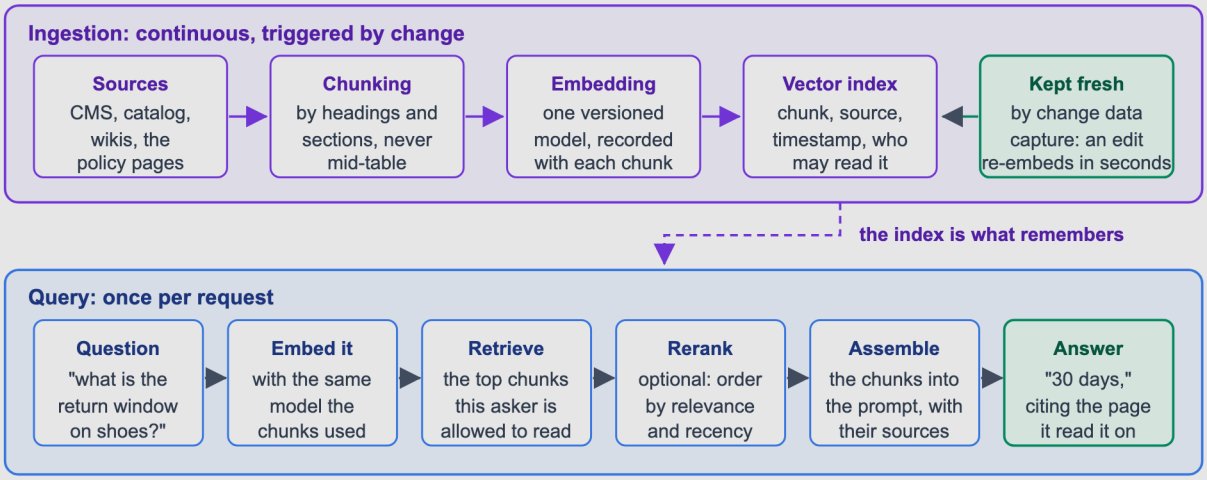
	Provider APIs	Serverless GPU	Own fleet
Cold starts	Provider's problem	Eased by platform	Yours
Cost at volume	Highest per token	Middle	Lowest if busy
Control / privacy	Least	Middle	Full
Fits	Most companies' start	Bursty, experiments	Big steady volume, privacy, custom models

Job	API gateway (inbound)	LLM gateway (egress)
Identity	AuthN of caller	Key custody toward providers
Fairness	Rate limits per client	Token budgets per team/feature
Routing	Paths → services	Logical models → providers
Discipline	No domain logic	No prompts

	Exact cache	Semantic cache	Provider prompt caching
Key	Identical string	Embedding proximity	Identical prefix
Reuses	Whole answer	Whole answer across wordings	KV state of prefix
False hits	Impossible	Managed risk	Impossible

Vector split	Fan-out	Recall	Fits
Even + scatter-gather	All N	Exact (per-shard ANN aside)	Global search, no filter
Tenant/metadata key	One shard	Exact within tenant	Multi-tenant (usual case)
Centroid (IVF)	2–3 clusters	Probabilistic dial	One huge latency-critical corpus

	RAG	Fine-tuning	Long context
Freshness	Seconds (CDC)	Weeks	Fresh (re-paste)
Citations	Built in	None	Unstructured
Access control	ACLs at retrieval	Impossible in weights	All or nothing
For	Knowledge	Behaviour	Small stable corpora



The model reasons; the index remembers. A policy edited this morning is answered correctly this afternoon without retraining anything, the answer points at the page it came from, and permissions are enforced at retrieval, where the system can still check them.

Fine-tuning is for behavior: tone, format, the way the answer is argued. This is for knowledge. They are not alternatives to each other.

In the interview

Q: Design a fraud detector/recommender — how do features stay consistent? **A:** Feature store: one definition, offline path with point-in-time joins, online streaming path, log-and-verify.

Q: Why do point-in-time joins matter? **A:** Leakage — current values on old labels teach the model the future; offline scores soar and production collapses.

Q: Why does batching help GPUs so much? **A:** Each token step streams all weights through the chip; sharing that stream across ~32 requests costs almost nothing extra.

Q: A 50-page paste spikes everyone's latency. **A:** Huge prefill hogs token steps — chunked prefill, input caps/tiering, prefill/decode pools at scale.

Q: LLM traffic goes up 5×. **A:** Queue-depth signal, warm pool = spike rate × boot time, calendar pre-warm, fast boots, degradation ladder, drain on scale-down, quota + spillover.

Q: Scale production GPUs to zero? **A:** Do the breakeven math: a small floor almost always beats multi-minute cold starts for real users; zero for dev.

Q: A dozen teams call providers directly — what do you build? **A:** An LLM gateway: key custody, hard token budgets, logical model routing with canaries, per-provider breakers and a fallback ladder, unbuffered streams, no prompts.

Q: Cut the LLM bill. **A:** Measure repetition; exact tier + guarded semantic cache at the gateway; provider prompt caching for shared prefixes; batch APIs for bulk.

Q: "Where's my order?" is the top question — cache it? **A:** No — per-user state; use a tool/API call and let the LLM format the result.

Q: RAG index outgrew one machine. **A:** Quantize first; shard by tenant if queries are scoped; otherwise scatter-gather with hedging; re-embed into a parallel index.

Q: Design a chatbot over company docs. **A:** CDC-fed ingestion with structure-aware chunking and ACL tags; query funnel cache → ACL-filtered retrieval → rerank → cited generation; recall@k and faithfulness evaluated separately.

Q: User asks about a doc they can't access. **A:** The retrieval filter excludes it — never enforce ACLs by prompt instruction.

Gotchas

- Two feature implementations "with careful testing"; training sets from current-state tables.
- Request-level batching for chat; hand-writing the token scheduler; mixing bulk and interactive on one fleet.
- CPU as GPU scaling signal; reactive-only; scale-to-zero in prod without math; killing generations on scale-down.
- Prompts in the gateway; no hard budget caps; buffering streams.
- Caching personalized answers; no false-hit monitoring; confusing semantic with prompt caching.
- Hash-sharding vectors by document ID and expecting keyed routing; re-embedding in place.
- Fine-tuning for freshness; ACLs by prompt; one quality metric for both RAG stages.

TL;DR

AI systems reuse known patterns under three constants — tokens as the unit, expensive slow-booting workers, similarity instead of equality: define features once, batch by token, scale GPUs on queue depth with warm pools, put a thin egress gateway in front of providers, cache by meaning carefully, shard vectors on filters, and build RAG as two pipelines with ACL-filtered retrieval.

12. Capstones

No new patterns — four full designs in the seven-stage interview format, then the course's selection tables.

Must-know

- **Seven-stage answer format** — (1) requirements & numbers → (2) five-questions pass → (3) annotated architecture → (4) 2–3 deep dives → (5) failure walk → (6) follow-ups → (7) pattern manifest.
- **Three rules interviewers judge** — **numbers before boxes**; **every pattern pays its price aloud**; **question four (failure) wins interviews**.
- **Seven threads** — (1) **skew**: hashing spreads keys, not weight → split, salt, pre-aggregate, dedicate, cache; (2) **same question, two authors** drift → define once; (3) **the log** — append-only history + position = durability and replay; (4) **budgets** — staleness and guarantees priced per question; (5) **idempotency under at-least-once**; (6) **ownership over agreement** — one owner + short leases replace locks; (7) **fail open vs fail closed**.

Design 1 — News Feed

- **Numbers:** 100M DAU; 500M reads/day ≈ 6,000/s (peak ~25,000/s); 10M posts/day ≈ 120/s (peak ~500/s); ~50:1 read:write; followers from 200 to 50M (skew is the main axis); new posts visible in ~10 s.

- **Core decision:** pull = 25,000 × 200 = 5M point reads/s (too costly); push = 120 × 200 = 24,000 appends/s (CQRS read model per user). **Hybrid:** push for most, pull-and-merge at read time for accounts above a measured ~100K-follower threshold.
- **Architecture:** posts sharded by author → **outbox** in the same transaction → partitioned queue → fan-out workers append IDs into Redis lists (cap ~800) on a **consistent-hash ring** → read = one list fetch + merge celebrity posts from a hot cache → hydrate via cache-aside → BFF, gateway, SSE hints.
- **Deep dives:** 100M × 800 × 16 B ≈ 1.3 TB (~40 Redis nodes) → LRU-evict inactive users (~3× less, rebuild by pull); celebrity stampede → single flight + early refresh; 10 s SLO = backlog-age autoscaling, batch appends per shard.
- **Failures:** fan-out lag → stale but valid; feed node dies → pull rebuild single-flighted; celebrity cache slow → breaker, partial feed; deleted post → filtered at hydration (tombstone).
- **One-liner:** "The feed is a disposable read model: push for ordinary accounts, pull for celebrities, rebuildable from the authoritative posts."

Design 2 — Ride-Sharing Dispatch

- **Numbers:** 10M rides/day ≈ 120/s (peak ~500/s); 2M drivers / 4 s = **500,000 location updates/s** (~1,000× business events, ephemeral); match within 10 s; a driver takes exactly one ride.
- **Architecture:** **geo-cells** (~1 km hexagons, H3/geohash) as the shard key; two-tier WebSocket gateways + backplane; in-memory per-cell index keeping only the latest position (**drop-oldest**, no sync durability); 1-in-10 sampled log for surge (tumbling windows) and training; ride as a **saga** with **pickup as the pivot**.
- **Deep dives:** exactly-one-winner via **single owner per cell matcher** + atomic claim with 10 s TTL + idempotent accept keyed by offer ID (**ownership replaces coordination**); airport hot cell → sequential matcher, split into 7 child cells, show queue position; hot path vs warm path, event time for GPS.
- **Failures:** matcher crash rebuilds from 4 s heartbeats (assignments in durable rides table); phone dies mid-trip — connection state ≠ trip state; gateway loss → jittered reconnect; payment failure → retry with keys then dunning (forward recovery).
- **One-liner:** "Drop superseded locations, give each driver one owner via TTL claims, run rides as sagas with pickup as the pivot, isolate cities as bulkheads."

Design 3 — Payment Platform

- **Numbers:** ~2,000 payments/s peak, 5M sellers, ~2 s authorization; **correctness, not throughput**, decides the architecture; financial decisions **fail closed**.
- **Architecture:** gateway rejects state changes without an **idempotency key**; **payment intent** state machine (`created → authorized → captured → recorded`) persisted before each side effect (intent log as WAL); card network behind timeouts/retries/breaker; webhooks never trusted alone; **event-sourced double-entry ledger** (each movement = debit + credit summing to zero; fixes = compensating entries); balances as projections with snapshots.
- **Deep dives:** one writer per account with **conditional appends** (hot fee account → batching + dedicated shard); internal transfers = **one journal event with both legs** (no 2PC); external payouts = sagas + idempotent bank instructions + reconciliation; daily **3-way match** (ledger, webhooks, settlement files).
- **Failures:** crash mid-capture → retry with same key; missing webhook → active query within hours, settlement file within a day; double refund → key + terminal state; bug wrote wrong amounts → reversing entries; network brownout → breaker, "nothing was charged".
- **One-liner:** "Money is an append-only double-entry history; balances are projections; internal transfers are single journal events; external boundaries use idempotent sagas plus reconciliation."

Design 4 — AI Copilot

- **Numbers:** 500K sellers, ~50 chat req/s peak, agent tasks seconds–minutes; it **acts** (edit listings, launch promos, message customers).
- **Architecture:** agent runtime as **durable execution** — each step journaled before it runs (resume, don't rerun); tool calls keyed by `(task_id, step_id)`; knowledge split: shared docs → RAG, private docs → tenant-sharded ACL-filtered indexes, live business data → **tools over CDC-fed CQRS projections**; actions classed as compensable / compensable within an undo window / **irreversible (pivot, human approval)**.
- **Deep dives:** runtime-enforced **token cap + action budget** + loop detection (breaker on reasoning); **prompt injection** — retrieved content is data, context-dependent tool allowlists, approval for irreversible

actions after reading untrusted content; testing via golden-task evals in a sandbox, flagged canaries, sampled human review.

- **Failures:** crash mid-task resumes at step N; provider outage → text degrades, **actions fail closed**; runaway loop → three independent bulkheads; wrong promo → designed compensation.
- **One-liner:** "Text may degrade, actions fail closed: durable step logs, runtime budgets, tools for state, ACL-filtered RAG for knowledge, irreversible actions behind approval."

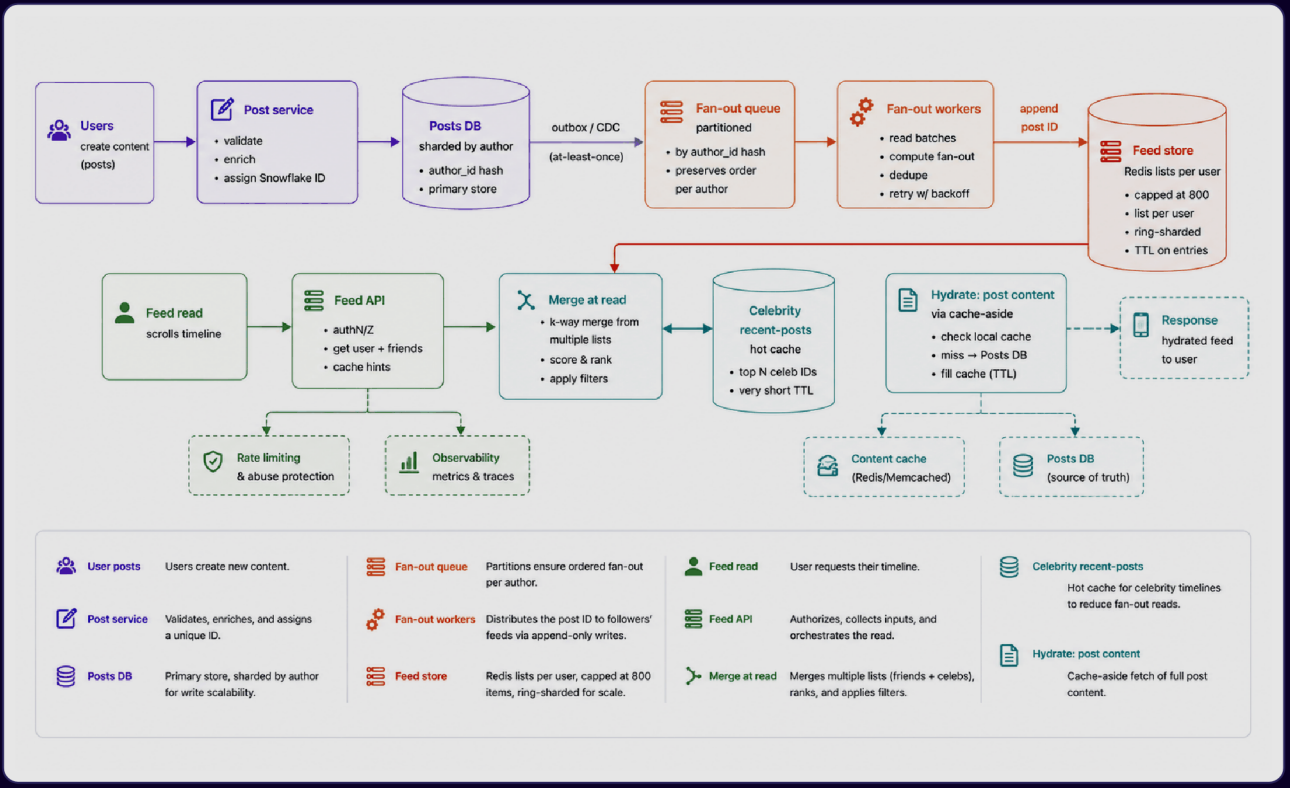
Key facts & numbers

Pattern selection — symptom → pattern

Symptom	Reach for
Checkout slows whenever analytics runs	Primary-replica (separate workloads)
DB too big or busy for one machine	Sharding
Adding one cache node reshuffled nearly every key	Consistent hashing
One user/product/tenant carries 40% of load	Hot-key escapes: split, salt, dedicate, cache
Crash lost acknowledged writes	Write-ahead log
"What was the state last Tuesday?" unanswerable	Event sourcing
One table serves five query shapes badly	CQRS
Stale data after bulk import / manual fix	CDC
Search index silently disagrees with DB	CDC + outbox
Synchronized cache expiry spikes the DB	Stampede prevention
A retry charged a customer twice	Idempotency
One slow dependency froze every thread	Timeout + circuit breaker
Retries made the outage worse	Backoff + jitter + budgets
One poison message jammed the queue	Dead letter queue
Dashboards green, one feature starving	Bulkhead
Feature broke → blank error page	Graceful degradation
Refund processed before its charge	Partitioned consumption (key by customer)
Producer outpaces consumer, memory climbing	Backpressure
Right answer, hours too late	Stream processing
Finance and dashboard disagree on revenue	Lambda/Kappa (one definition)
Billing drifts after crashes/deploys	Exactly-once semantics
Two datacenters both think they're primary	Quorum (majorities)
Money moves between systems with no shared DB	Saga
Can't tell which service is slow	Distributed tracing
One release broke every user at once	Canary + feature flags
Model great offline, bad in production	Feature store
GPUs at 8% utilization	Model serving & batching
Spike outran the 10-minute GPU boot	GPU auto-scaling (warm pools, calendars)
Scattered provider keys, $40K runaway	LLM gateway
40% of LLM queries are rewordings	Semantic caching
Chatbot quotes last quarter's policy	RAG pipeline

Decision rules

Choice	Rule
Push vs pull (feed)	Push for most; pull above a measured follower threshold
SSE vs WebSocket	SSE unless the client must send mid-stream
Queue vs pub/sub	One consumer per job → queue; many independent consumers of a fact → pub/sub
Cache strategy	Start cache-aside; read-through if the library fits; never write-behind for money
Vertical vs horizontal	One big-enough box before a distributed fleet
2PC vs saga	2PC only inside one trust boundary; sagas across; prefer ownership or a single journal
Batch vs streaming	Staleness budget decides; stay batch if it meets the deadline
Lambda vs Kappa	One definition per metric; Kappa if retention covers reprocessing
Idempotent vs transactional sink	Keyed upserts first; transactions when outputs + offsets must move atomically
Raw CDC vs outbox	Raw for your own infra; outbox for other teams; never dual-write
RAG vs fine-tune vs long context	Knowledge / behaviour / small stable corpora — combinable
Own GPUs vs provider APIs	APIs until volume, latency or privacy justify a fleet



In the interview

Q: Why not always pull the feed? A: 25K loads/s × 200 = 5M scattered reads/s versus 24K appends/s for push; the 50:1 ratio justifies moving the join to write time.

Q: Make the feed ranked. A: Fan-out delivers candidates (~2,000), a read-time ranker scores them; never rank at write time (personal, changes hourly).

Q: Why can't two matchers offer the same driver? A: Each driver has exactly one owner (its cell's matcher) and assignments are atomic TTL claims — no locks or 2PC.

Q: Why not a global matching service? A: Matching is local; global adds latency, a hot surface and a worldwide blast radius — cities are bulkheads.

Q: Why not a balances table you UPDATE? **A:** Loses audit history, hides errors the sum-to-zero rule exposes, needs locking; keep balances as projections of the ledger.

Q: Where's your 2PC in payments? **A:** None internally (one journal, one event per transfer); externally 2PC can't cross companies → sagas + idempotency + reconciliation.

Q: GDPR deletion on an immutable ledger? **A:** Crypto-shred personal fields; financial records retained under regulatory exemption.

Q: How is a copilot different from a RAG chatbot? **A:** It acts — durable execution, action budgets, reversibility classes, approval gates, sandboxed evals.

Q: "Just let the agent do everything." **A:** Autonomy per action type: compensable actions run with undo windows; irreversible ones need approval by default.

Gotchas

- Drawing boxes before numbers → the math removes components.
- Ranking at write time; deleting posts eagerly across 50M lists.
- Locks or 2PC for driver assignment; letting warm-path durability eat the hot path's budget.
- 2PC with the bank; balance UPDATEs as the source of truth; trusting webhooks alone.
- ACLs or safety via prompt instructions; letting actions degrade to a fallback model.

TL;DR

Diagnose the real problem (fan-out skew, local ownership, correctness, safe actions), compose patterns through the five questions, say every cost aloud, and win on the failure walk — then use the symptom table and decision rules to pick the right pattern fast.

13. Case Studies: Building Blocks

Six classic "design X" prompts that other systems are built on; no new patterns — each tests two or three you already know.

Must-know

- **Case format (modules 13–17)** — requirements → **numbers** → five-questions pass → **API + data model** → annotated architecture → 2–4 deep dives → failure walk → trade-offs → follow-ups → pattern manifest + one-liner.
- **Recurring moves** — **precompute the read**; **hash to place, range to scan**; **coordinate rarely** (leases, not per-request locks); **append, then compact**; **probabilistic structures** (Bloom, SimHash) buy memory with a stated error; **duplicates are normal — make them harmless**.

Unique ID Generator

- **Requirements:** unique, compact (64-bit), roughly time-sortable IDs from hundreds of machines; no central counter on the write path.
- **Key numbers:** 1 sign bit + **41-bit ms timestamp (~69.7 years)** + **10-bit worker (1,024)** + **12-bit sequence (4,096/ms/worker)**.
- **Core design:** computed in-process; worker numbers **leased** from etcd/ZooKeeper; DB unique constraint as last net.
- **Hardest part:** clock rollback (wait or refuse), duplicate worker IDs (leases + fencing), sequence overflow (wait for next ms).
- **One-liner:** "Time + worker + sequence in 64 bits, generated locally; the only coordination is leasing worker numbers."

URL Shortener

- **Requirements:** create short links, redirect fast, analytics, custom aliases, abuse checks.
- **Key numbers:** 100M links/day (1K/s), 10:1 reads (10K/s, 50K peak); 365B links in 10 years → **7 base-62 chars**, ~180 TB.
- **Core design:** counter from **leased ranges** → reversible scramble → base 62 (no collisions); **302** redirect via CDN + cache-aside (negative caching); store sharded by code; clicks to a queue.

- **Hardest part:** code generation without collisions or guessability; 302 (control, analytics) vs 301 (cacheable forever).
- **One-liner:** "Range-leased counter → base 62; redirects from CDN and cache; analytics off the critical path."

Distributed Key-Value Store (Dynamo-style)

- **Requirements:** always writable through node failures; tunable consistency; no lost writes.
- **Key numbers:** 10 TB × 3 replicas, 1M ops/s → ~2.3M replica ops/s → **~60 nodes** of 500 GB (throughput, not size, sets it).
- **Core design:** consistent hashing + vnodes; preference list = next 3 distinct machines; **N/W/R** quorums (3/2/2); sloppy quorum + **hinted handoff**; **version vectors** → siblings; gossip; read repair + **Merkle** anti-entropy; LSM node (commit log → memtable → SSTables + Bloom).
- **Hardest part:** availability vs linearizability; deletes (tombstones + grace period; repair more often than the grace period).
- **One-liner:** "Ring + vnodes, W=2/R=2, sloppy quorum and hints for failures, version vectors instead of lost writes, Merkle trees for drift."

Distributed Message Queue (Kafka-style)

- **Requirements:** durable, replayable, ordered (per key) log at millions of messages/s; many independent consumers.
- **Key numbers:** 1 GB/s in, 7-day retention → ~900 TB replicated → **~30 brokers**; 128 partitions on the big topic.
- **Core design:** partition = segmented append-only log + offset/time indexes; leader + ISR, high watermark; `acks=all`, min ISR 2, no unclean election; pull consumers commit offsets after processing; idempotent producer + transactions.
- **Hardest part:** durability vs latency; order only per partition; partition count caps parallelism.
- **One-liner:** "Partitioned, replicated append-only log; `acks=all` + min-ISR 2 survives a broker; pull + offsets = backpressure and replay."

Web Crawler

- **Requirements:** crawl billions of pages politely, dedupe URLs and content, avoid traps, recrawl by change rate.
- **Key numbers:** 1B pages/month ≈ 400/s (design **1,000/s**), ~310 Mbit/s, ~25 TB/month compressed; ≥2,000 hosts in flight; **10B URLs → 12 GB Bloom filter at 1%**.
- **Core design:** frontier = **front queues (priority)** + **back queues (one host each)** + min-heap of next-allowed times; hosts partitioned across machines by consistent hashing; Bloom for "URL seen?", exact hash + **SimHash** for content.
- **Hardest part:** politeness at scale; traps (URL length/depth caps, per-host budgets); freshness vs coverage.
- **One-liner:** "Priority plus per-host politeness queues, host-partitioned workers, Bloom and SimHash dedup, recrawl by measured change rate."

Search Autocomplete

- **Requirements:** top-k suggestions per keystroke, fast, safe (no offensive or personal suggestions).
- **Key numbers:** 600M requests/day (~20K/s peak), **p99 < 20 ms** server-side; ~20M queries → **10–20 GB trie** (fits in memory).
- **Core design:** trie with **top-k cached per node** (O(prefix length) lookup); daily batch: logs → decayed counts → filters → build → validate → **blue/green swap** to in-memory replicas; small stream job for trending; browser + CDN caching, debounce.
- **Hardest part:** freshness vs simplicity; filtering what the product "says"; shard by prefix range only when memory forces it.
- **One-liner:** "Precompute everything: top-k per trie node, rebuilt offline, swapped atomically, cached at browser and CDN."

Key facts & numbers

Case	The number that shapes it	Main trade-off
ID generator	4,096 IDs/ms/worker × 1,024 workers, 69.7 years	Clock/worker-ID risk vs central-counter bottleneck
URL shortener	10K redirects/s vs 1K creates/s; 7 chars	302 control vs 301 cacheability
KV store	1M ops/s → ~2.3M replica ops/s → ~60 nodes	Availability vs linearizability
Message queue	1 GB/s, 7 days → ~900 TB	`acks=all` durability vs latency; order per partition
Web crawler	1,000 pages/s; 12 GB Bloom for 10B URLs	Freshness vs coverage; false positives vs memory
Autocomplete	20K req/s, < 20 ms; 10–20 GB trie	Freshness vs safety and simplicity

Code to remember

```
def snowflake(ts_ms, worker, seq, epoch_ms):
    return ((ts_ms - epoch_ms) << 22) | (worker << 12) | seq   # 41 | 10 | 12 bits
```

In the interview

Q: Are Snowflake IDs strictly ordered? **A:** Only by millisecond, then by worker — not by true arrival; strict global order needs one sequencer or consensus.

Q: Why not UUIDs? **A:** Fine if 16 bytes is acceptable — UUID v7 is time-ordered; 64-bit IDs win in many large indexes or 64-bit-only systems.

Q: Same long URL submitted twice — same code? **A:** Only if wanted: look up `hash(long_url)` per owner; different owners usually get separate codes (separate stats and delete rights).

Q: Make the KV store strongly consistent. **A:** One leader per partition replicated with Raft/Paxos (etcd, TiKV, Spanner); you lose availability during elections and on the minority side.

Q: Why not a database table as a queue? **A:** Fine at low volume; at 1M msg/s row locks, deletes and index churn dominate — a log is append-only with one offset per consumer.

Q: More consumers than partitions? **A:** Extra consumers sit idle; partition count is the ceiling, so over-provision up front.

Q: Avoid two crawler machines hitting one site? **A:** Host-based partitioning — each host owned by exactly one machine, enforced by the hash, not locks.

Q: Personalise autocomplete. **A:** Merge the global top-k with the user's recent searches at request time; never cache personal answers publicly (`Cache-Control: private`).

Gotchas

- Sequential IDs in anything that must be unguessable (password-reset links) → separate random token.
- Hash-and-truncate short codes without conditional insert + retry → silent collisions.
- "Queues deliver exactly once" → at-least-once + idempotent consumers (+ transactions inside the log).
- Tombstones purged before repair runs → deleted data resurrects.
- Treating every discovered URL as equally worth fetching → score pages, budget by quality.
- Running a model per keystroke without candidate generation → rank only the top ~50.

TL;DR

Building blocks are two or three known patterns under one prompt: say the shaping number first (bits, QPS, TB, URLs), then the one core mechanism and its main trade-off.

14. Case Studies: Messaging and Media

Chat, email, video, file sync and live streaming — separate blobs from metadata, sync through ordered logs and cursors.

Must-know

- **Across all five** — separate **blobs from metadata**; sync through **ordered logs + cursors**; notifications are **hints**; every retry is **idempotent**; shard by the key every query carries.

Chat System

- **Requirements:** 1:1 and group messages, offline catch-up, all devices, receipts, presence, push; p99 < 500 ms online; no lost acknowledged messages; per-conversation total order; no visible duplicates.
- **Key numbers:** 500M DAU × 40 = **20B msgs/day ≈ 230K/s (700K/s peak)**; ~2M deliveries/s peak; **150M sockets** → ~3,000 gateways at 50K each; heartbeats **5M pings/s** (answered at the gateway).
- **Core design:** stateless-about-data gateways + session registry + backplane; **one sequencer per conversation** (gap-free `seq`); message log keyed `(conv, bucket)`; per-user **inbox log with device cursors**; push for offline.
- **Hardest part:** order without clocks, duplicates (`client_msg_id` + dedupe by seq), reconnect sync, presence (pull + TTL, never broadcast).
- **One-liner:** "Gateways hold sockets and own nothing; one sequencer per conversation; ack after durable append; at-least-once with dedupe; one cursor per device."

Email Service

- **Requirements:** receive/send internet mail, thread/label/search, spam filtering, IMAP/SMTP; **never lose accepted mail**; inbox p99 < 300 ms, search < 1 s.
- **Key numbers:** 1B users; accepted inbound **40B/day ≈ 460K/s (1.4M/s peak)**; ~30 KB/message → **1.2 PB/day (~440 PB/year)**; metadata ~1 KB = 3% of bytes, ~100% of queries.
- **Core design:** SMTP edge → **replicated spool before** `250 OK` → async spam/malware/index pipeline → content-addressed blob store + **mailbox-sharded** metadata + per-mailbox search; outbound queue per destination domain, SPF/DKIM/DMARC, retries for days.
- **Hardest part:** the `250 OK` boundary (at-least-once + dedupe), deliverability and reputation, per-mailbox search.
- **One-liner:** "Spool before 250, process async, shard by mailbox, blobs content-addressed, outbound queue per domain with authentication."

Video Platform (YouTube-like)

- **Requirements:** resumable upload, multi-resolution transcode, ABR playback; start < 2 s, rebuffering < 0.5%; watchable within minutes; originals never lost.
- **Key numbers:** 15B min/day → ~20M peak streams × 3 Mbps ≈ **60 Tbps egress**; 500 h uploaded/min ≈ 50 uploads/s; **~300K cores** for the H.264 ladder; ~2.4 EB/year stored.
- **Core design:** resumable direct-to-object-storage upload → **chunked parallel transcoding DAG** of idempotent tasks → immutable HLS/DASH segments + manifests → origin shield → CDN → ABR players.
- **Hardest part:** egress cost and the long tail (tiered CDN, ISP caches); expensive codecs (VP9/AV1) only where views repay them.
- **One-liner:** "Upload resumably, transcode as a parallel DAG of small idempotent tasks, publish immutable segments, let ABR players pull through a tiered CDN."

File Sync (Drive/Dropbox-like)

- **Requirements:** any file size, sync across devices, offline edits, sharing, versions; durability above all; changes visible in ~10 s; conflicts visible, never silent.
- **Key numbers:** 500M users, ~300M devices; 2.5 EB logical (dedupe saves 20–30%); **1 trillion file entries (~500 TB metadata)**; ~70K commits/s peak.
- **Core design:** client **content-defined chunking** + hashing → block service (upload blocks first) → **conditional commit** to a per-namespace journal (metadata last) → hint via long-poll → pull with cursor → fetch missing blocks.

- **Hardest part:** dedupe (with its privacy catch), conflicts and tree moves, fan-out on huge shared folders.
- **One-liner:** "Content-addressed blocks so only new ones move; blocks first, conditional metadata commit last; hint then pull by cursor; conflicted copies, never silent overwrites."

Live Streaming

- **Requirements:** real-time ABR viewing, DVR window, recording to VOD; glass-to-glass ~3–5 s (creators) to 10–20 s (sports); rebuffering < 1%; the big event must not fail.
- **Key numbers:** 100K channels × 6 Mbps = 600 Gbps ingest; event **25M viewers × 3 Mbps = 75 Tbps**; 2 s segments → **~12M req/s for a handful of URLs**; "play" spike hits the API tier.
- **Core design:** RTMP/SRT ingest PoPs → **pinned per-channel real-time transcode + LL-HLS/DASH packaging** → origin → shields → edges with **request collapsing** → players.
- **Hardest part:** latency budget (segment/part size vs buffer), hot-object herd, survival (reconnect windows, leases, **1+1 pipelines**, multi-CDN, pre-scaling).
- **One-liner:** "Ingest near the broadcaster, a pinned real-time pipeline per channel, short segments with partial chunks, fan out via tiered CDN with request collapsing; pre-scale big events."

Key facts & numbers

Case	Center of gravity	Source of truth	Shard key
Chat	150M sockets, per-conversation order	Message log + inbox log	`(conv, bucket)`, user
Email	Never lose accepted mail; per-mailbox queries	Spool → blobs + metadata	Mailbox
Video	60 Tbps egress, transcode CPU	Object storage segments	Video
File sync	70K atomic commits/s on 1T entries	Per-namespace journal	Namespace
Live	One hot object × 25M viewers	Per-channel pipeline	Channel

In the interview

Q: How do you order group-chat messages? **A:** One sequencer per conversation assigns gap-free numbers; clients order by `seq`, never by device clocks.

Q: A phone reconnects after 2 hours offline. **A:** It sends its inbox cursor; the server returns everything after it in order; the socket resumes streaming.

Q: When is an email "accepted"? **A:** Only after it is durable in a replicated spool — then `250 OK`; everything after is async and idempotent.

Q: Why is the video design really a CDN problem? **A:** ~60 Tbps of egress can't come from one site; segments are immutable, so edges and shields cache them.

Q: Why content-defined chunking for sync? **A:** Fixed-size blocks shift after an insert and every later block changes; content-defined boundaries keep unchanged blocks identical so only new ones upload.

Q: Two devices edited the same file offline. **A:** Conditional commit on the base revision fails for the second; it becomes a **conflicted copy** — keep both.

Q: 25M viewers request the same new segment. **A:** Request collapsing at edges and shields: one fetch upstream per location, everyone else waits on it.

Gotchas

- Broadcasting presence to every friend → pull + TTL.
- Acking a chat message before it is durable → loss on crash.
- Rejecting spam after `250 OK` without a plan → bounce storms (reject at SMTP time where possible).
- Transcoding a whole video as one task → split into chunk tasks, retry per task.
- Treating the push/long-poll hint as the data → hints trigger a cursor pull.
- One pipeline for the World Cup final → 1+1 redundancy, multi-CDN, pre-scaled control plane.

TL;DR

Messaging and media designs split cheap blobs from precious metadata, move changes through ordered logs read by cursors, and push bytes through CDNs; the hard parts are order, duplicates and the hot object.

15. Case Studies: Location and Real-Time

Turn 2-D closeness into 1-D keys, fan out moving points, precompute maps, and rank millions in O(log N).

Must-know

- **Cell index** – geohash (base-32 string prefix = cell), quadtree (split dense cells), **S2** (cells on a cube-projected sphere), **H3** (hexagons: equal-distance neighbours). All turn "near me" into "same or neighbouring cell".
- **Boundary problem** – a neighbour just across a cell edge is missed → always search the cell **plus its 8 neighbours**, then filter by exact distance.

Proximity Service (nearby places)

- **Requirements:** find places within a radius, fast; place writes are rare.
- **Key numbers:** **100M places → ~5 GB index**; 20K searches/s → ~6 replicas per region.
- **Core design:** cell ID + 9-cell search + exact-distance filter; **replicated in-memory index** refreshed by CDC from the places DB.
- **Hardest part:** uneven density (adaptive cells), boundaries, keeping writes off the search path.
- **One-liner:** "2-D closeness → 1-D cell key; search cell + 8 neighbours, filter by distance; the index is a few GB, so replicate it in memory, don't shard it."

Nearby Friends

- **Requirements:** see friends' live locations within a radius; lossy is fine, privacy strict.
- **Key numbers:** 10M online, **~333K updates/s in**, **~13M deliveries/s** inside; ~100 gateways, ~130 pub/sub nodes.
- **Core design:** WebSocket gateways; latest location in a **TTL cache**; **one pub/sub channel per user**; online friends' gateways subscribe and **filter by distance at the receiver**.
- **Hardest part:** stateful resizing of the pub/sub ring, reconnect storms, privacy checks that fail closed.
- **One-liner:** "Latest location in memory with a TTL, published on the user's channel, filtered by distance on the receiving side – lossy by design, strict on privacy."

Maps and Navigation

- **Requirements:** render maps, route A→B, live traffic and ETA, rerouting.
- **Key numbers:** **~92B** possible raster tiles to zoom 18 (why vector tiles + over-zoom); **~1.2M GPS points/s**; ~5K routes/s.
- **Core design:** batch-built **versioned vector tiles** on a CDN; in-memory road graph with **Contraction Hierarchies** or multi-level partitions; stream **map matching** → per-segment speeds; ETA = segment sum + ML correction; historical profiles as fallback.
- **Hardest part:** routing on stale traffic, herding everyone onto one road, GPS-trace privacy.
- **One-liner:** "Precompute what doesn't change by the minute (tiles, shortcut graph); stream live traffic into per-segment speeds; fall back to history when it lags."

Real-Time Gaming Leaderboard

- **Requirements:** update scores, top-N, "my rank", monthly boards.
- **Key numbers:** **25M players ≈ 2.5 GB**, ~3K writes/s → one node + replica.
- **Core design:** match results are the truth; **Redis sorted set** (hash + skip list) per month = O(log N) update, rank, range; idempotent updates by match ID; tie-break encoded in the score.
- **Hardest part:** retries double-counting, arbitrary tie order, global rank once sharded (scatter-gather).
- **One-liner:** "Match results as truth, a sorted set per month as the read model; shard by hash only when one node is outgrown."

Key facts & numbers

	Geohash	Quadtree	S2	H3
Cell shape	Rectangle (varies with latitude)	Square, adaptive	Quadrilateral on a cube face (Hilbert order)	Hexagon (6 equidistant neighbours)
Adapts to density	No	**Yes**	Via mixed-level coverings	No
Storage	String/int column + normal index	Custom in-memory tree	64-bit int	64-bit int
Radius query	9-cell trick	Walk the tree	Built-in region coverer	k-ring of hexagons
Best for	Simple, DB-friendly; Redis GEO	Very uneven density	Precise coverings	Movement, pricing grids

In the interview

Q: Why not a plain SQL `WHERE lat BETWEEN … AND lng BETWEEN …`? **A:** Two 1-D indexes can use only one range efficiently; a cell key makes it one indexed lookup.

Q: Shard or replicate the proximity index? **A:** Replicate — ~5 GB fits in memory; shard only if it outgrows a node.

Q: Why filter nearby friends at the receiver? **A:** The publisher doesn't know who is near; the subscriber's gateway knows its user's position and drops far updates cheaply.

Q: Why not Dijkstra on the whole road graph? **A:** Too slow at continent scale; Contraction Hierarchies precompute shortcuts so a query touches a tiny part of the graph.

Q: How do you avoid rerouting everyone onto the same side road? **A:** Feed predicted load back into routing and spread alternatives; don't treat live speeds as independent of your own suggestions.

Q: Two players tie. **A:** Encode the tie-break (earlier time wins) into the score so the order is deterministic.

Gotchas

- Searching only the user's own cell → the boundary problem.
- Persisting every GPS ping durably → keep latest only (TTL), sample for analytics.
- Raster tiles for every zoom → vector tiles + over-zoom.
- Leaderboard updates without a match ID → double counts on retry.
- Sharding a 2.5 GB sorted set on day one → one node + replica.

TL;DR

Location problems become 1-D keys (cells) plus an exact filter; real-time location is lossy pub/sub with receiver-side filtering; maps precompute and stream; leaderboards are sorted sets.

16. Case Studies: Infrastructure

Object storage, metrics, job scheduling, counters and collaborative editing — the platforms other teams build on.

Must-know

- **Erasure coding RS(k, m)** — split into k data + m parity fragments; any k rebuild the object; RS(8,4) = **1.5× overhead** vs 3× for replication, at the cost of repair bandwidth.
- **Cardinality** — number of distinct label combinations (series); the real budget of a metrics system.
- **Fencing token** — a monotonically increasing number handed out with each lease; storage rejects writes carrying an older token.
- **HyperLogLog** — estimates distinct counts in a few KB with ~1% error.

S3-like Object Storage

- **Requirements:** PUT/GET/DELETE/LIST by key, huge durability, strong read-after-write.
- **Key numbers:** 100 PB, 50B objects (~2 MB avg, skewed); 20K PUT/s, 200K GET/s; **~25 TB metadata**; 3× replication → 15,000 disks vs **RS(8,4) → 7,500** (halves hardware).
- **Core design:** immutable **chunks** on data nodes + strongly consistent, **range-sharded metadata**; the **metadata row write is the commit point**; fragments across racks/AZs; small objects packed into volume files; deletes = tombstones + compaction.
- **Hardest part:** durability = placement + **repair speed**; replicate while hot, encode when cold.
- **One-liner:** "A strongly consistent metadata index pointing at immutable, erasure-coded chunks across failure domains; the metadata commit is when an object exists."

Metrics Monitoring and Alerting

- **Requirements:** ingest host/app metrics, dashboards, alert rules, long retention.
- **Key numbers:** 100K hosts × 1,000 series = **100M series**, every 10 s = **10M samples/s**; Gorilla compression **~1.37 bytes/sample** (13.8 → ~1.2 TB/day); ~400 GB RAM for series buffers; 50K rules ≈ 1.7K queries/s.
- **Core design:** write-heavy TSDB: memory + WAL for recent data, immutable blocks on object storage, downsampling; delta-of-delta timestamps + XOR values; alerting on **SLO burn rate**; group/inhibit/silence; runbook links.
- **Hardest part:** cardinality (police unbounded labels like `user_id`); monitoring the monitor (**dead-man's switch** from outside).
- **One-liner:** "A compressed time-series store whose budget is cardinality, plus alerts on user-facing symptoms – itself watched from outside."

Distributed Job Scheduler

- **Requirements:** one-off and cron jobs, priorities, retries, at-least-once with effectively-once effects.
- **Key numbers:** 100M executions/day ≈ 1,160/s, **~20K/s at the top of the hour**; 2 s avg → ~2,300 busy slots (Little's law); 3 TB hot history.
- **Core design:** **job table is the truth** (partitioned by job ID, indexed by `next_run_at`); schedulers **lease partitions**, claim due rows with `SKIP LOCKED`, write the next cron run in the same transaction; weighted priority queues → workers under leases.
- **Hardest part:** "exactly once" = at-least-once + **idempotency keys + fencing tokens**; starvation (weights, aging, quotas); top-of-hour spikes (jitter).
- **One-liner:** "A partitioned, time-indexed job table leased to schedulers, feeding priority queues drained by leased workers; idempotency + fencing make at-least-once effectively once."

Distributed Counter (likes and views)

- **Requirements:** like/unlike, show counts, count views and unique viewers, multi-region.
- **Key numbers:** 10B likes/day ≈ 115K/s (400K/s peak); 1.2M views/s; **viral post 50K likes/s** vs a few thousand/s per locked row; 500K count reads/s from cache.
- **Core design:** likes are **facts** (set of who liked); count is a derived, recomputable cache; **batch in a stream aggregator**; **shard hot counters**; views approximate, **HyperLogLog** for uniques; **CRDT counters** across regions.
- **Hardest part:** the hot row; double counting; bots.
- **One-liner:** "Likes are a set, the count is a cached derivative: batch per post, shard viral counters, HLL for uniques, CRDTs across regions."

Collaborative Editor (Docs-style)

- **Requirements:** many people edit one document live, see each other's cursors, history and undo, offline edits.
- **Key numbers:** **~2M ops/s** peak (client batches every 50–100 ms); ~17 TB/day raw op log → compacted into snapshots; 1B docs ≈ 50 TB of snapshots.
- **Core design:** apply locally first; converge with **OT** (transform against one server order) or **CRDT** (unique IDs + tombstones, ops commute); **one owner server per document** (consistent hashing + lease + fencing); append-only op log + snapshots; ack after durable append; presence as ephemeral pub/sub.
- **Hardest part:** convergence *and* intent preservation under concurrency.

- **One-liner:** "Local-first edits merged by OT (central order) or CRDT (commuting unique-ID ops), with an event-sourced op log and snapshots per document owner."

Key facts & numbers

	OT	CRDT
Core idea	Rewrite positions of concurrent ops	Unique immutable IDs per element; ops commute
Central server?	In practice yes (defines the order)	No; offline/peer-to-peer natural
Metadata cost	Low	Higher (IDs + tombstones)
Hard part	Transform function for every op pair	Tombstone GC, metadata size
Known users	Google Docs (Jupiter lineage), Etherpad	Yjs, Automerge; CRDT ideas in Figma

	3× replication	RS(8,4) erasure coding
Raw for 100 PB	300 PB	150 PB
Survives	2 lost copies	Any 4 lost fragments
Cost	Disk	CPU + repair bandwidth, slower small reads

In the interview

Q: When is an object "written"? **A:** When its metadata row commits — chunks written before are invisible garbage until then, collected later.

Q: Replication or erasure coding? **A:** Replicate hot, small or recently written data; erasure-code cold data for ~1.5× overhead.

Q: An engineer adds `user_id` as a metric label. **A:** Series multiply by millions and ingest RAM explodes — use logs/traces for per-user data; enforce label limits.

Q: Why page on burn rate, not CPU? **A:** Users feel symptoms, not causes; burn-rate alerts fire when the error budget is being spent too fast.

Q: Two schedulers both think they own a partition. **A:** Leases with fencing tokens — the job store rejects the stale owner's writes.

Q: 50K likes/s on one post. **A:** Batch increments per post in a stream aggregator and shard the counter into N sub-counters summed on read.

Q: OT or CRDT? **A:** OT when a central server orders edits anyway (simpler, compact); CRDT for offline or peer-to-peer at the cost of metadata.

Gotchas

- Durability math that ignores repair time → correlated failures win.
- Metrics system alerting only through itself → external dead-man's switch.
- Cron at `00:00` for everyone → jitter and spread.
- `UPDATE posts SET likes = likes + 1` for a viral post → hot row; store facts, derive counts.
- Last-write-wins in a text editor → lost keystrokes; OT or CRDT.

TL;DR

Infrastructure designs separate a small strongly consistent index from bulk data, treat facts as truth and counts/queues as derived, and make at-least-once safe with idempotency and fencing.

17. Case Studies: Money and Reservations

Correctness under concurrency: how money moves, the last hotel room, a 100K/s wallet and a microsecond exchange.

Must-know

- **Authorization vs capture vs settlement** — auth = the issuer's promise to pay; capture = asking for the money; clearing = agreeing who owes what; **settlement** = money actually moves. Never credit on authorization.
- **Rails** — cards: real-time auth, batch clearing/settlement, **chargebacks for months**; **ACH**: batch, can be **returned**; wires and real-time rails (FedNow, RTP, UPI): instant and **final**.
- **Reconciliation** — compare your ledger with the provider's/bank's independent report daily; the only proof both sides agree.
- **Double-entry** — every movement is debits + credits summing to zero; balances are projections.
- **Exactly-once effect** — no rail offers exactly-once delivery: idempotency key stored in the **same transaction** as the state change.

How Money Moves (primer)

- **Core design:** a payment is a **state machine** (authorized → captured → settled → reversed); outbound calls carry idempotency keys; daily reconciliation; integer cents; tokens instead of card numbers (PCI scope).
- **Hardest part:** "paid" means different things on different rails; webhooks are at-least-once and may never arrive.
- **One-liner:** "An authorization is a promise, not money: model the states, key every instruction, reconcile daily."

Hotel Reservation

- **Requirements:** search by city/dates, book, pay, cancel; never oversell beyond a chosen limit.
- **Key numbers:** 1M rooms; ~233K bookings/day, **~30/s peak**; look-to-book 100:1 → ~3,000 searches/s; inventory **36.5M rows ≈ 2 GB** (fits one DB).
- **Core design:** per-night inventory rows on the hotel's shard; **one short transaction** of conditional `UPDATE`s (in date order, avoiding deadlocks); idempotency key on the reservation; **hold → pay → confirm** saga with a sweeper; search from a cache.
- **Hardest part:** the last room (the DB row is the lock); overbooking as a computed number per room type and night; never hold a lock across the payment call.
- **One-liner:** "The database is the lock: conditional per-night updates in one short transaction, an idempotency key, hold → pay → confirm."

Digital Wallet

- **Requirements:** balances, top-ups, instant P2P transfers, withdrawals, full audit; p99 < ~300 ms.
- **Key numbers:** **100K transfers/s** peak × ~5 row writes = 500K writes/s; ~5,000 transfers/s per shard → **20 shards** (provision 40–64); ~140 TB/year.
- **Core design:** **double-entry journal** as the truth, balances as projections with snapshots; shard by account; same-shard = one local transaction; **cross-shard = saga through a transit account** (not 2PC); idempotency key with each entry; deduping consumers.
- **Hardest part:** cross-shard transfers, exactly-once, hot accounts (split into sub-accounts, dedicated shard).
- **One-liner:** "A double-entry journal is the truth; shard by account; cross-shard transfers are sagas through a transit account; exactly-once = idempotent writes + deduping consumers."

Stock Exchange

- **Requirements:** limit/market orders and cancels, fair price-time priority, market data, no lost acknowledged orders.
- **Key numbers:** 8,000 symbols; 500M messages/day ≈ 21K/s avg, **~500K/s peak** (~50K/s busiest symbol); internal budget **~20 µs**; market data ~320 Mbit/s → **UDP multicast**.
- **Core design:** gateways → in-memory risk → **sequencer** (stamp, persist, replicate) → **one single-threaded matching engine per symbol partition** → output log → brokers, market data, post-trade; standby **replays the same input log**.
- **Hardest part:** one total order per book, determinism, tail latency (no GC pauses, no network hops on the hot path).
- **One-liner:** "One sequencer, one thread per book, log before act, replicate the input not the state."

Key facts & numbers

	Hotel	Wallet	Exchange
Peak writes	~30 bookings/s	~100,000 transfers/s	~500,000 msgs/s
Source of truth	Inventory + reservation rows	Append-only journal	Sequenced input log
Scaling unit	Hotel shard	Account shard	Symbol partition
Concurrency control	Row locks / conditional updates	Conditional debit / single writer per shard	Single thread, no locks

In the interview

Q: Why not a Redis lock around the room type? **A:** The row lock already gives mutual exclusion tied to the write's transaction; a separate lock can expire mid-write and adds a dependency.

Q: Show "only 2 rooms left" honestly? **A:** From the cache, labelled approximate; truth is enforced at booking.

Q: How is a transit account different from 2PC? **A:** Nothing is locked across shards; each step commits alone and the in-between state is visible money that reconciliation can see.

Q: Why can't you trust the webhook? **A:** At-least-once and possibly never delivered — reconcile against the settlement report.

Q: User tops up by ACH and withdraws instantly to a card. **A:** The ACH debit can be returned days later — hold funds for the return window or limit by risk score.

Q: Scale the matching engine for one hot symbol? **A:** No — price-time priority needs one order per book; scale across symbols; one symbol fits one thread.

Q: Orders in flight during exchange failover? **A:** Acknowledged ones are in the replicated log and replayed; unacknowledged ones are resent, `clOrdId` rejects duplicates.

Gotchas

- Crediting on authorization → chargebacks and returns become losses.
- Holding a DB lock across a payment call → hold → pay → confirm with expiry.
- Balance `UPDATE`s as the truth → journal + projections.
- 2PC across shards or companies → sagas + idempotency + reconciliation.
- Floating-point money or prices → integer cents / ticks.

TL;DR

Money and reservations are correctness problems: the row or the journal is the truth, every write carries an idempotency key, cross-boundary work is a saga plus reconciliation, and fairness comes from one sequence.

18. Client and Mobile System Design

Design the app side: a flaky network, a battery, OS limits and releases you can't hot-fix; the server is one linked box.

Must-know

- **Backend vs client interview** — constraints are one device (network, battery, memory, OS); estimate MB on disk, MB/day, requests per session, frames per second.
- **Nine building blocks** — layers (UI → state → repository → local DB / network / sync) · offline-first and sync · caching and persistence · networking and pagination · push · background work · battery/data · releases and flags · observability.
- **Push** — APNs/FCM; payload ≤ **4 KB** (APNs); may be late, duplicated, collapsed or dropped → a **hint**, never the data.
- **Frame budget** — **16.7 ms** at 60 Hz: no disk, network, JSON parsing or image decoding on the main thread.

- **Background work** – WorkManager (Android) / BGTask, background `URLSession` (iOS); Doze and background limits mean "poll every minute" never runs on schedule.

Framework rules (graded in every client answer)

- **UI reads local state; the network writes into it.** Instant launch, offline support, consistent screens.
- **Every network call has an offline story** (slow, failed, succeeded after the app was killed).
- **Every write gets an idempotency key on the device**; durable **outbox** drained by a background job.
- **Sockets for the foreground only**; push + cursor sync otherwise. **Assume you cannot hot-fix**: phased rollout, kill-switch flags, minimum supported version.

App cards

- **News feed app** – ~150 posts/session, 20/page → 8 pages ≈ 240 KB JSON; 90 images at display size ≈ **9 MB** vs ~150 MB for originals; one decoded 1080×1350 image ≈ 5.8 MB. *Design:* DB-backed paging with cursors, two-tier (memory + disk) image cache, optimistic likes via outbox. *Hardest:* smooth scrolling and data use. *One-liner:* "The screen observes a local table filled page by page; images at display size, cached in two tiers; every action an optimistic write drained idempotently."
- **Chat app** – sent in < 500 ms p95, nothing typed ever lost. *Design:* local outbox + client message ID; server per-conversation `seq` defines order; WebSocket in foreground, push + sync in background; gap detection by sequence. *One-liner:* "Outbox first, client ID for safe retries, server sequence for order, socket only in the foreground."
- **Stock trading app** – server **conflates** to ≤ 4 updates/s/symbol; 50 symbols streamed ≈ 43 MB/h vs **~9 MB/h** for the ~10 visible rows. *Design:* snapshot + conflated stream for visible symbols, price age shown; orders on a separate path with client order ID; on doubt, **reconcile before retry**. *One-liner:* "Stream only what's visible, label staleness, never resend an order without asking the server what happened."
- **Hotel reservation app** – search page ≈ 520 KB with thumbnails. *Design:* cancellable, cached, paged search keyed by the full query; server **quote with expiry**; idempotent tokenised booking; trips pinned offline. *One-liner:* "Search is a cached cancellable read; booking is a quoted idempotent write the app can always reconcile."
- **File sync app** – 50K files ≈ 15 MB of local metadata (content on demand); 2 GB video at 10 Mbit/s ≈ 27 min → **8 MB resumable chunks**. *Design:* local metadata tree + change cursor; remote/local/synced trees to tell who changed what; hashed chunked transfers as background jobs; conflicted copies. *One-liner:* "Whole metadata tree locally, a synced base tree to plan against, resumable hashed chunks, keep both on conflict."
- **Video streaming app** – 480p ≈ 450 MB/h, 720p ≈ 1.1 GB/h, 1080p ≈ 2.25 GB/h; 4 s segments. *Design:* HLS/DASH from a CDN; **buffer-based ABR**; start low, preload the likely next video; downloads as background jobs with expiring offline licences; data-saver default on cellular. *One-liner:* "Pick each segment's quality mainly from the buffer, start low, preload next, downloads as background jobs."

Key facts & numbers

Symptom on the device	Reach for
Spinner on every launch; blank offline	Offline-first: UI observes the local DB
Actions typed offline vanish when app is killed	Durable outbox + background job
Duplicate messages/orders after retries	Client-generated idempotency key
Items repeat or skip while scrolling	Cursor pagination + dedupe by ID
Scroll stutter when images load	Off-main-thread decode, sized URLs, recycled rows, memory LRU
Battery drain in background	Close sockets; push as a hint; batch requests
Messages missing after reconnect	Sync token / sequence numbers with gap detection
"Did my payment go through?" after a timeout	Reconcile by idempotency key before retrying
A bad release crashed 5% of users	Phased rollout + kill switch + minimum version
"Slow for users" but server metrics fine	Client-side metrics and traces

In the interview

Q: Does the UI read from the network or the database? **A:** The database; the network only updates it — that gives instant launch, offline and consistent screens.

Q: The user sends a message in a tunnel and swipes the app away. **A:** It is already in the outbox (same local transaction as the message row); a background job or next launch sends it with its client ID.

Q: Why not poll every minute in the background? **A:** The OS won't run it on schedule (Doze, background limits) and each wake keeps the radio in high power; use push as a hint plus sync.

Q: An order request times out. Retry? **A:** Not blindly — query by client order ID first; resend with the same ID only if the server has no record.

Q: How do you ship safely when you can't roll back an app? **A:** Phased rollout, features behind remote flags with kill switches, a minimum-version gate, and an API that still works for years-old clients.

Q: What do you estimate in a mobile interview? **A:** Bytes per screen and session, cache sizes on disk and in memory, requests per session, frame budget — not cluster QPS.

Gotchas

- Screen bound straight to a network response → blank offline, inconsistent screens.
- Full-resolution images in a list → 15× data and OOM; request display size.
- WebSocket kept open in the background → battery drain; push + sync.
- Trusting push payloads as data → treat as a hint, fetch by cursor.
- Retrying writes without a client-generated key → duplicates.
- Shipping without flags or a minimum version → a crash you can't recall.

TL;DR

Client design = local state as the UI's truth, an offline story for every call, idempotent outbox writes, foreground-only sockets with push as a hint, and releases guarded by flags because you can't hot-fix.

Last-minute checklist

- Open with requirements and **numbers**: users, RPS (avg and peak = 3–5× avg), data size, read:write, cost of downtime. Day ≈ 100,000 s.
- Announce the agenda: **Move → Live → Fast → Fail → Grow**, then refinements (consistency, entry point, operations).
- Every arrow: does the caller wait? If not → queue/event, and consumers are **idempotent**.
- Jobs go on queues, facts get published; schemas are public contracts (additive only).
- Reads bottleneck → cache + replicas; writes/size → shard on a key you justify aloud; resizes → consistent hashing.
- Hashing spreads keys, not weight — name the **hot key** before the interviewer does.
- Cache: cache-aside, **DEL on write**, TTL per data type, judge by misses, single-flight hot keys; never cache money decisions.
- Every call: **timeout** (2–3× p99, propagated budget) → **jittered retries** at one layer → **idempotency key** → **breaker** → **fallback**.
- Bulkheads for success-shaped floods; DLQ with an owner and an alarm; pause ordered streams instead of dead-lettering.
- Stateless servers, least-connections LB, shallow health checks, auto-scale on requests/latency/backlog age, **max = downstream capacity**.
- Pools small and shared (PgBouncer); 50 servers × 20 connections = 1,000.
- Cross-service workflows → **saga** (durable log, compensations, pivot last); 2PC only inside one trust boundary.
- Leaderless: **W + R > N**; vector clocks detect conflicts, LWW is a data-loss policy.
- Entry point: CDN (fingerprint assets), thin gateway (AuthN at entrance, AuthZ in services), token-bucket limits with 429 + Retry-After, cursor pagination, additive versioning with Sunset dates.
- Observability: metrics alarm, traces localize, logs explain; tail-sample errors and outliers.

- Change safely: blue-green / canary vs concurrent baseline / flags (fail static, done = deleted); **expand-contract** for schemas.
- Pipelines: staleness budget picks batch vs stream; event time + watermarks; one metric definition; CDC not dual-writes; exactly-once only where auditors look; bounded buffers.
- AI: tokens are the unit; batch by token; scale GPUs on queue depth with warm pools; thin LLM gateway (no prompts); semantic cache with audits; RAG with ACL-filtered retrieval.
- Money and auth **fail closed**; optional features degrade; AI text may degrade, actions may not.
- Case prompts (ch. 13–18): state the **shaping number** first (sockets, TB, ops/s, egress), then the one core mechanism, then the hardest part – e.g. chat = per-conversation sequencer; wallet = double-entry journal + transit-account sagas; exchange = one sequencer, one thread per book.
- Client prompts: UI reads local state, every call has an offline story, idempotency key on the device, sockets only in the foreground, flags because you can't hot-fix.
- Close with the **failure walk**: for each dependency – slow, down, duplicated – what does the user see?